第 2 章 密码学基础

本章学习要点：

- 掌握密码学的基本概念和发展过程；
- 掌握对称密码算法的基本思想，了解主要算法的基本内容；
- 掌握公钥密码算法的基本思想，了解主要算法的基本内容；
- 了解 Hash 函数、数字签名、密钥管理技术的基本内容。

信息安全的主要任务是研究计算机系统和网络空间中信息的保护方法，密码学是信息安全的重要研究领域之一，可以说密码学是保障信息安全的核心基础。随着计算机与通信技术的快速发展，人们对网络环境的依赖程度日渐加深，信息发布、数据传输、资源共享等应用广泛覆盖政治、军事、金融、教育、医疗等各个领域，相关的安全、防护等问题日益受到关注和重视。

以电子邮件应用场景为例，与传统的信息传输模式相比，电子邮件因具有很强的时效性、便捷性成为日常交流、电子商务、电子政务中广泛使用的通信方式。然而，在电子邮件带来便利的同时，信息泄露、欺骗假冒等安全问题也给用户的利益带来隐患和威胁。因此，为了满足用户的安全需求，电子邮件系统需要借助加密算法、Hash 函数等密码工具来保障用户的敏感数据、重要文件在存储和传输过程中不被泄露、篡改或伪造，利用数字签名、密钥管理等技术来防止信息发送者假冒他人发送邮件或发送后进行抵赖。

密码学能够为实现机密性、完整性、可用性、认证性、不可否认性等信息安全的安全目标提供理论依据和技术支撑。本章将从密码学中的基本概念入手，在尽量避免烦琐数学公式的前提下，对密码学的核心思想、典型算法、发展现状等进行简要介绍。

2.1 密码学概述

2.1.1 密码体制基本概念

密码学(cryptology)是研究如何隐秘地传递信息的学科，是结合数学、计算机、信息论等学科的一门综合性、交叉性学科，其首要目的是隐藏信息的含义，而不是隐藏信息的存在。

2.1.1.1 密码体制的组成

密码学起源于保密通信技术，基本思想就是对信息进行伪装。伪装前的信息称为明文，通常用 p(plaintext)或 m(message)表示；伪装后的消息称为密文，通常用 c(ciphertext)表示。这种对信息的伪装可以表示成一种可逆的数学变换，从明文到密文的变换称为加密(encryption)，从密文到明文的变换称为解密(decryption)。加密和解密都是在密钥(key)的控制下进行的。一个密码体制(cryptosystem)一般包括以下五个组成部分。

（1）明文空间 M，它是全体明文 m 的集合。

（2）密文空间 C，它是全体密文 c 的集合。

（3）密钥空间 K，它是全体密钥 k 的集合，其中每一个密钥 k 均由加密密钥 k_e 和解密密钥 k_d 组成，即 $k=(k_e,k_d)$。

（4）加密算法 E，是在密钥控制下将明文消息从 M 对应到 C 的一种变换，即 $c=E(k_e,m)$。

（5）解密算法 D，是在密钥控制下将密文消息从 C 对应到 M 的一种变换，即 $m=D(k_d,c)$。

基于密码技术的保密通信的基本模型如图 2.1 所示。

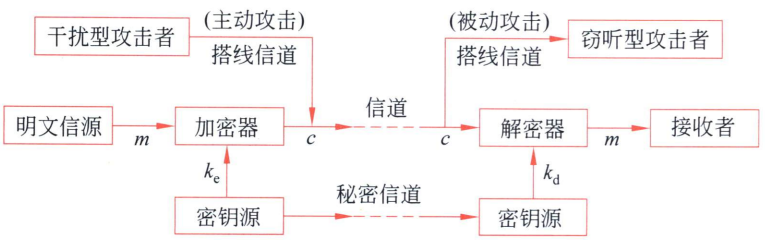

图 2.1 保密通信的基本模型

密码学包括密码编码学（cryptography）和密码分析学（cryptanalysis）两部分。密码编码学主要研究如何设计编码，使得信息编码后除指定接收者外的其他人都不能读懂。密码分析学主要研究如何攻击密码系统，实现加密消息的破译或消息的伪造。随着科学技术的创新和应用，这两个分支相互对立又相互促进，不断推动着密码学的发展。

2.1.1.2 密码体制的分类

密码体制的加、解密过程必须要在密钥的控制下完成。根据加、解密密钥设计策略的不同，可将密码体制分为对称密码体制和非对称密码体制。

1. 对称密码体制

如果一个密码体制中的加密密钥 k_e 和解密密钥 k_d 相同，或者由其中一个密钥很容易推算出另一个密钥，则称该密码体制为对称密码体制或单钥密码体制。因为在使用过程中，密钥必须严格保密，所以也被称为秘密密钥密码体制。

对称密码体制具有安全、高效、经济等特点，因此被广泛应用。依据处理数据的方式，对称密码体制通常又分为分组密码（block cipher）和序列密码（stream cipher）。分组密码是以定长的明/密文块（如 DES 密码以 64 位一组）为处理对象；序列密码则是以单个位为对象，对明/密文位逐个进行处理，因此也被称为流密码。

对称密码体制主要用于对信息进行保密，实现信息的机密性。它的优点是加密和解密处理效率高，密钥长度相对较短，一般情况下加密后密文和明文长度相同。但是，对称密码体制也存在一些固有的缺陷，如需要安全通道分发密钥、保密通信的用户数量多时密钥量大难于管理、难以解决不可否认性等。

2. 非对称密码体制

1976 年，Diffie 和 Hellmen 发表了具有里程碑意义的论文《密码学的新方向》，提出了非对称密码的思想，即加密过程和解密过程使用“不同”的密钥来完成。这里的“不同”指在计算上不能由加密密钥 k_e 推出解密密钥 k_d，那么将 k_e 公开不会损害 k_d 的安全。因此，这种可以将 k_e 公开的密码体制通常也被称为公钥密码或双密钥密码体制。一般把加密密钥 k_e 称为公钥，解密密钥 k_d 称为私钥。

公钥密码体制的提出解决了对称密码体制的固有缺陷，它不仅可以保障信息的机密性，还可以对信息进行数字签名，具有认证性和抗否认性的功能。不过，公钥密码体制与对称密码体制相比，其设计所依赖的数学计算较复杂，因而加、解密效率较低。在达到同样安全强度时，公钥密码通常所需的密钥位数较多，并且加密产生的密文长度通常会大于明文长度。因此，在保密通信过程中通常是用对称密码来进行大量数据的加密，而用公钥密码来传输少量数据，如对称密码所使用的密钥。

2.1.1.3 密码体制设计原则

一个实用的密码体制的设计应该遵守以下原则。

(1) 密码算法安全强度高。就是攻击者根据截获的密文或某些已知明文密文对，要确定密钥或者任意明文在计算上不可行。

(2) 密码体制的安全性不应依赖加密算法的保密性，而应取决于可随时改变的密钥。即使密码分析者知道所用的加密体制，也无助于其用来推导出明文或密钥。

(3) 密钥空间应足够大，使试图通过穷举密钥空间进行搜索的方式在计算上不可行。密钥空间中不同密钥的个数称为密码体制的密钥量，它是影响密码体制安全性的重要因素。

(4) 既易于实现又便于使用，使加密函数和解密函数都可以高效地计算。

其中第2条是著名的柯克霍夫(Kerckhoffs)原则，是由荷兰密码学家奥古斯特·柯克霍夫于1883年在其名著《军事密码学》中提出的。如果密码体制的安全强度依赖攻击者不知道的密码算法，那么这个密码体制最终必定失败。柯克霍夫原则指出密码算法应该是公开的。密码算法的公开不仅有利于增加密码算法的安全性，还有利于密码技术的推广应用、增加用户使用的信心，更有利于密码技术的发展。

密码技术是保障网络安全的核心技术，密码算法和密码产品的自主可控是确保国家信息安全的重中之重。自1999年发布《商用密码管理条例》以来，我国的密码研究和密码应用水平快速发展。国产密码算法：SM2/SM9数字签名算法、SM3密码杂凑算法、祖冲之密码算法、SM9标识加密算法、SM4分组密码算法先后被纳入了ISO/IEC国际标准。为了规范密码应用和管理，促进我国密码事业发展，保障网络与信息安全，维护国家安全和社会公共利益，保护公民、法人和其他组织的合法权益，《中华人民共和国密码法》由全国人大常委会表决通过并于2020年1月1日起开始施行。

2.1.2 密码学的发展历史

人类对密码的研究和应用已有几千年的历史，自从有了战争，就有了保密通信，也就有了密码的应用。密码学的发展经历大致可以分为三个阶段：古典密码时期、近代密码时期和现代密码时期。

1. 古典密码时期

学者一般认为古典密码时期是从古代到19世纪末，这个阶段长达数千年。这一时期可视为科学密码学的前期，此时期的密码技术可以说是一种艺术。密码的设计和分析通常凭借的是直觉和信念，而不是推理和证明。

据史料记载，大约在公元前700年，古希腊军队用一种称为密码棒的圆木棍来进行保密通信，其使用方法是：把长带子状羊皮纸缠绕在圆木棍上，然后在上面写字；解下羊皮纸后，上面只有杂乱无章的字符，只有再次将羊皮纸以同样的方式缠绕到同样粗细的棍子上，才能看出所写的内容。在使用密码棒时，明文的字母不发生变化，只是顺序被打乱了，后来人们把这类密

码归结为换位密码。

大约在公元前100年，古罗马的执政官和军队统帅恺撒发明了一种把所有的字母按字母表顺序循环移位的文字加密方法被称为恺撒密码。他将英文字母按字母表的顺序构成一个字母序列链，然后将最后一个字母与第一个字母相连成环。加密的方法是将明文中的每个字母用其后的第三个字母代替。解密时，只需把密文中每个字母用其前的第三个字母代替即得明文。例如，使用恺撒密码对明文字符串 It is a secret 加密，结果为 LWLVDVHFUHW。

将明文字母表中的每个字母用密文字母表中的相应字母来替换，这类密码被称为代换密码。代换密码根据一个字符被替换成固定的一个字符或可能被替换成多个字符，可分为单表代换密码和多表代换密码。恺撒密码就是一种单表代换密码，其明密文字母对照表可以用表2.1表示。

表2.1 恺撒密码明密文字母对照表[①]

m	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o	p	q	r	s	t	u	v	w	x	y	z
	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25
	+3 mod 26																									
c	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	0	1	2
	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C

恺撒密码用数学语言则可以表示为

$$M=C=\{x \mid x \in [0,25] \text{ 且 } x \in \mathbf{Z}\}$$

$$k_e=k_d=3$$

$$E(k_e,m)=(m+3)\bmod 26$$

$$D(k_d,c)=(c-3)\bmod 26$$

这个时期的经典案例还有棋盘密码、美国南北战争时期军队中使用过的栅栏密码等。由于这个时期生产力低下，产生的许多密码体制都是以纸笔或简单器械实现加、解密的，它们的基本技巧都是进行较简单的代换、换位或是两者的结合。现在看来，古典密码体制大多数比较简单而且容易破译。这一时期的密码主要应用于军事、政治和外交领域。古典密码加解密方法主要基于手工完成，密文信息一般通过人（信使）来传递，此时期也被称为密码学发展的手工阶段。

2. 近代密码时期

近代密码时期从20世纪初期到20世纪50年代末。19世纪的工业革命为使用更加复杂的密码技术提供了条件，频繁的战争加速了密码技术的发展。

在第一次世界大战中，传统密码的应用达到了顶峰。1837年，美国人莫尔斯发明了电报。1896年前后，意大利发明家马可尼和俄国物理学家波波夫发明了无线电报，人类从此进入电子通信的时代。无线电报能快速、方便地进行远距离收发信息，很快成为军事上的主要通信手段。但是，无线电报是一种广播式通信，任何人包括敌人都能够接收电报信号。为了防止机密信息的泄露，电报文件的加密变得至关重要。

随着科学和工业的飞速发展，在第二次世界大战中，密码学的发展远远超过了之前的任何时期。参战各国认识到密码是决定战争胜负的关键，纷纷研制和采用先进的密码设备，建立最

① 为加以区分，这里明文用小写字母表示，密文用大写字母表示。

严密的密码安全体系。越来越多的数学家不断加入密码研究队伍，大量的数学和统计学知识被应用于密码分析，加密原理从传统的单表代换发展到复杂度大幅提高的多表代换，基于机械和电气原理的加密和解密装置全面取代了以往的手工密码，人类从此进入机械密码时代，德国的 Engima、英国的 TYPEX、日本的 RED 和 PURPLE 等都是此阶段著名的密码机。

在这个时期，密码设计者设计出了一些利用电动机械设备实现信息加密、解密操作的密码方法，采用电报机发送加密的信息。这个时期虽然加解密技术和设备有了很大的进步，但是还没有形成密码学理论，加解密的主要原理仍然是代换、换位及两者的结合。在密码破译方面，破解原理基于字母的统计频率特性。对于复杂的多表代换古典密码加密方法，利用密文的重合指数方法与密文中字母统计规律相结合，同样可以破译。

这个时期还有一个非常重要的密码体制就是一次一密密码体制。美国电话电报公司的 Gillbert Vernam 在 1917 年为电报通信设计了一种非常方便的密码，即在对明文加密前，首先将明文编码为由 0、1 组成的序列，加密时用明文与密钥进行模 2 相加，解密时将密文再与密钥模 2 相加即可。例如，密钥为 10010 00101 时对明文位串加密结果如下：

明文：10001 11000

密文：00011 11101

该加密变换和解密变换可以用数学语言表述如下（其中⊕表示模 2 加）：

$$M=\{m=(m_0,m_1,\cdots,m_i,\cdots)\mid m_i=0\text{ 或 }1\}$$

$$C=\{c=(c_0,c_1,\cdots,c_i,\cdots)\mid c_i=0\text{ 或 }1\}$$

$$K=\{k=k_e=k_d=(k_0,k_1,\cdots,k_i,\cdots)\mid k_i=0\text{ 或 }1\}$$

$$E(k,m)=(m_0\oplus k_0,m_1\oplus k_1,\cdots,m_i\oplus k_i,\cdots)$$

$$D(k,c)=(c_0\oplus k_0,c_1\oplus k_1,\cdots,c_i\oplus k_i,\cdots)$$

一次一密密码加解密操作简单，但为了保证安全性，该密码体制需要使用真正随机的密钥。通信双方在密钥的产生、分配、存储、同步等环节实现起来都非常困难，致使它成为一种难于实际应用的密码。虽然一次一密实用性方面较差，但它在理论上有非常重要的价值，并且为序列密码的诞生奠定了基础。关于序列密码的相关内容将在 2.2.2 节中给出进一步介绍。

3. 现代密码时期

随着计算机的诞生，人们对密码体制的设计及安全性有了更高的要求。1949 年，Shannon 发表了《保密系统的通信理论》，将信息论引入密码学的研究，把密码学置于数学基础之上，为密码编码学和密码分析学奠定了坚实的理论基础，使密码技术由艺术变成了科学。

1976 年，美国斯坦福大学的密码专家 Diffie 和 Hellman 发表了划时代的论文——《密码学的新方向》，提出了一个崭新的思想。该思想中不仅密码算法可以公开，用于加密消息的密钥也可以公开，这就是著名的公钥密码体制思想。这一思想标志着公钥密码体制的诞生，是密码发展的重要里程碑，Diffie 和 Hellman 也因该成果于 2015 年获得了图灵奖。

20 世纪 70 年代中期以前的密码学研究基本上是秘密进行的，主要用于军事、政府、外交等重要部门。密码学的真正蓬勃发展和广泛应用是从 20 世纪 70 年代中期开始的。1977 年美国颁布了数据加密标准（Date Encryption Standard，DES），揭开了密码学的神秘面纱，使密码学得以在商业等民用领域广泛应用。1978 年，美国麻省理工学院的 Rivest、Shamir 和 Adleman 基于数论中的大整数因子分解困难问题，提出了第一个实用的公钥密码体制——RSA 公钥密码。此外，比较著名的算法还有 1994 年美国联邦政府颁布的密钥托管加密标准（EES）和数字签名标准（DSS），2001 年美国联邦政府颁布的高级加密标准（AES）等。

由于计算机技术及相关学科的进步和发展，现代密码学的任务已经不局限于保密通信功能，而是扩展到身份认证、数字签名、密钥管理、密码协议等更多信息安全相关内的内容。同时，密码研究也出现了一些新兴的、交叉性的理论，如混沌密码、DNA 密码、量子密码、格密码等。

从密码学的发展历史可以看出，整个密码学的发展过程是从简单到复杂、从不完善到较为完善、从具有单一功能到具有多种功能、从传统理论到多学科交叉融合的过程。如今的电子支付系统、5G 通信技术、区块链、云计算和人工智能等都离不开密码技术的保护和支持。在国家的大力推动下，密码技术不仅与高科技发展相融合，而且还处于新兴产业、数字经济发展的前沿。

2.1.3 密码体制的安全性

密码分析学是伴随着密码编码学的产生而产生的，它是研究如何分析或破解各种密码体制的一门科学。密码分析也被称为密码攻击，指非授权者在不知道解密密钥的条件下对密文进行分析，试图得到明文或密钥的过程。密码攻击和解密的相似之处在于都是设法将密文还原成明文的过程，但攻击者和消息接收者所具备的条件是不同的。密码分析者的任务是恢复尽可能多的明文，或者最好能推算出解密密钥，这样就很容易解出被加密的信息。

密码分析可以发现密码体制设计的弱点，常用的分析攻击方式主要有以下 3 种。

（1）穷举分析攻击：密码分析者通过试遍所有的密钥来进行破译。穷举分析攻击又被称为蛮力攻击，指攻击者依次尝试所有可能的密钥对所截获的密文进行解密，直至得到正确的明文。1997 年 6 月 18 日，美国科罗拉多州 Rocket Verser 工作小组宣布，通过网络利用数万台计算机历时 4 个多月以穷举分析攻击方式攻破了 DES。

（2）统计分析攻击：密码分析者通过分析密文和明文的统计规律来破译密码。统计分析攻击在历史上为破译密码做出过极大的贡献。许多古典密码都可以通过分析密文字母和字母组的频率及其统计参数而破译。例如，在英语里，字母 e 是英文文本中最常用的字母，字母组合 th 是英文文本中最常用的字母组合。在简单的替换密码中，每个字母只是简单地被替换成另一个字母，那么在密文中出现频率最高的字母就最有可能是 e，出现频率最高的字母组合就最有可能是 th。抵抗统计分析攻击的方式是在密文中消除明文的统计特性。

（3）数学分析攻击：密码分析者针对加密算法的数学特征和密码学特征，通过数学求解的方法来设法找到相应的解密变换。为对抗这种攻击，应该选用具有坚实的数学基础和足够复杂的加密算法。

一般地，衡量密码体制安全性的方法有以下 3 种。

（1）计算安全性，也称实际保密性。如果一种密码系统最有效的攻击算法至少是指数时间的，则称这个密码体制是计算安全的。在实际中，人们说一个密码系统是计算上安全的，意思是利用已有的最好方法破译该系统所需要的努力超过了攻击者的破译能力（如时间、空间和资金等资源）。

（2）可证明安全性。Shannon 曾指出，设计一个安全的密码本质上是要寻找一个难解的问题。例如，RSA 密码可以归结为大整数因数分解问题。如果密码体制的安全性可以归结为某个数学困难问题，则称其是可证明安全的。

（3）无条件安全性或完善保密性。假设存在一个具有无限计算能力的攻击者，如果密码体制无法被这样的攻击者攻破，则称其为无条件安全。Shannon 证明了一次一密密码具有无条件安全性，即从密文中得不到关于明文或密钥的任何信息。这里需要强调的是：一次一密密码达

到无条件安全的前提必须是密钥是真正的随机数，不是通过计算机程序生成的伪随机数。

密码学领域存在一个很重要的事实："如果许多聪明人都不能解决的问题，那么它可能不会很快得到解决。"这意味很多加密算法的安全性并没有在理论上得到严格的证明，只是这种算法思想出来以后，经过许多人许多年的攻击并没有发现其弱点，没有找到攻击它的有效方法，从而认为它是安全的。

2.2 对称密码

对称密码体制建立在通信双方共享密钥的基础上，算法使用的加密密钥和解密密钥相同或由其中一个很容易推导出另一个。自1977年美国颁布DES密码算法作为美国数据加密标准以来，对称密码体制迅速发展，得到了世界各国的关注和普遍应用。对称密码体制包括分组密码和序列密码两大类，本节将对这两类密码体制依次进行介绍。

2.2.1 分组密码

分组密码是现代密码学的重要体制之一，广泛应用于数据的保密传输、加密存储等场合。分组密码是将明文消息编码后的序列划分成固定大小的组，每组明文分别在密钥的控制下得到对应的密文序列。下面以明文编码成二进制为例进行描述。

设 n 是分组密码每个明文的分组长度，$k=(k_0,k_1,\cdots,k_{t-1})$ 是密钥，分组密码的示意图如图2.2所示，其中 $x=(x_0,x_1,\cdots,x_{n-1})$ 为明文，$y=(y_0,y_1,\cdots,y_{m-1})$ 为加密算法在密钥 k 控制下对应产生的密文，$x_i,y_j\in\{0,1\}$，$0\leqslant i\leqslant n-1$，$0\leqslant j\leqslant m-1$。

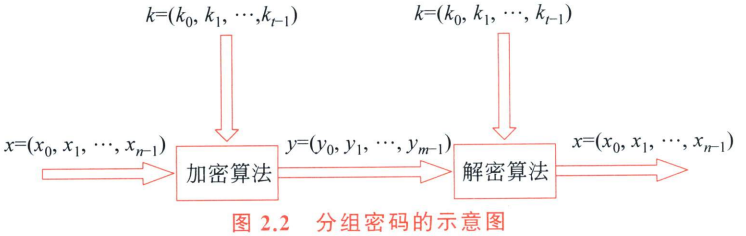

图2.2 分组密码的示意图

若 $n<m$，则分组密码对明文加密后有数据扩展；若 $n>m$，则分组密码对明文加密后有数据压缩；若 $n=m$，则分组密码对明文加密后既无数据扩展也无数据压缩，大部分分组密码属于这种情况。

分组密码的本质就是由密钥 k 控制的、从明文空间 M（长为 n 的位串集合）到密文空间 C（长为 m 的位串集合）的一个一对一映射。

2.2.1.1 分组密码的基本原理

扩散和混淆是Shannon提出的设计分组密码体制的两种基本方法，其目的是抵抗攻击者对密码体制的统计分析。**扩散**（diffusion）就是将明文的统计特性散布到密文中去，使得明文的每一位影响密文中多位的值，也就是说，密文中每一位均受明文中多位影响。在分组密码中，可对数据重复执行某个置换，再对这一置换作用于一个函数，可获得扩散。**混淆**（confusion）就是使密文和密钥之间的统计关系变得尽可能复杂，使得攻击者即使获取了关于密文的一些统计特性，也无法推测出密钥。使用复杂的代换算法可以得到预期的混淆效果。古典密码中最基本的变换就是代换和置换，分组密码都离不开这两种最基本的变换。

从安全性角度考虑，如果攻击者知道明文的某些统计特性（如消息中不同字母出现的频率、可能出现的特定单词或短语），而且这些统计特性以某种方式在密文中反映出来，攻击者就有可能得出加密密钥或其一部分，或者得出包含加密密钥的一个可能的密钥集合。在分组密码的设计中，充分利用扩散和混淆，可以有效地抵抗攻击者从密文的统计特性推测出真正明文或密钥。因而，扩散和混淆成为设计现代分组密码的基础。在设计分组密码时通常选择某些较简单的受密钥控制的密码变换，利用乘积和迭代的方法就可以取得比较好的扩散和混淆效果。

例如，S_1 和 S_2 的乘积密码体制定义为 $S_1 \times S_2 = (M_1 \times M_2, C_1 \times C_2, K_1 \times K_2, E_1 \times E_2, D_1 \times D_2)$，其中 $S_1 = (M_1, C_1, K_1, E_1, D_1)$ 和 $S_2 = (M_2, C_2, K_2, E_2, D_2)$ 是两个密码体制。在实际应用中，明文空间和密文空间往往是相同的，即 $M_1 = M_2 = C_1 = C_2$，此时乘积密码体制 $S_1 \times S_2$ 可简化表示为 $S_1 \times S_2 = (M_1 \times M_2, K_1 \times K_2, E_1 \times E_2, D_1 \times D_2)$。对任意明文 $x \in M$ 和密钥 $k = (k_1, k_2) \in K_1 \times K_2$，加密变换的数学形式可表示为 $E_1 \times E_2(k_1, k_2, x) = E_2(k_2, E_1(k_1, x))$。类似地，对任意的密文 $y \in C$ 和密钥 k，解密变换则可以表示为 $D_1 \times D_2(k_1, k_2, y) = D_1(k_1, D_2(k_2, y))$。

分组密码中两种常见的迭代结构是 Feistel 结构（图 2.3）和 SP 网络（代换-置换网络）。

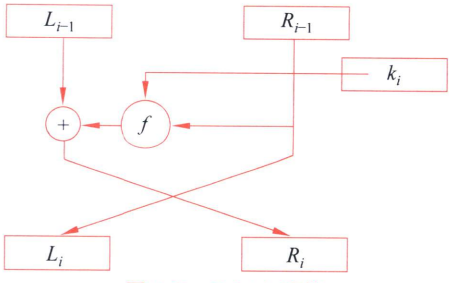

图 2.3 Feistel 结构

Feistel 结构可分为平衡的 Feistel 结构和非平衡的 Feistel 结构。平衡的 Feistel 结构密码算法中，通常取分组长度 n 为偶数，然后将每组输入分长度为 $n/2$ 的左、右两部分。定义一个迭代的分组密码算法，其第 i 轮的输出 R_i 和 L_i 取决于前一轮的输入 R_{i-1} 和 L_{i-1}（图 2.3），数学公式可表述为

$$\begin{cases} L_i = R_{i-1} \\ R_i = L_{i-1} \oplus f(R_{i-1}, K_i) \end{cases}$$

式中：K_i 是第 i 轮使用的子密钥；f 是轮函数。Feistel 结构的分组密码加密过程与解密过程相似，其加密过程最后一轮没有进行左右交换，其优点就是可以利用同一个算法来实现加密和解密，DES 算法的设计就采用了这种结构。当然，随着计算能力的提高，分组密码的分组长度逐步加长，Feistel 结构的弊端也会逐步展现出来。分组长度的增加意味着轮函数 f 规模的增加，而构造大规模的轮函数又是比较困难的，因此非平衡 Feistel 结构被提出用于对较大的明文分组进行加密。将明文分为 n 个运算字，进行 n 次迭代就能将明文全部覆盖一遍。非平衡 Feistel 结构能够改善混淆并提高扩散效率，其最大的优点是能够直接重用过去的轮函数。我国国密算法 SM4 分组密码算法采用的就是非平衡的 Feistel 结构。

SP 网络是由代换（也称 S 盒）和置换（也称 P 盒）交替进行多次而形成的变化网络。代换起到混淆的作用，置换经过多轮迭代并同代换结合，能产生扩散作用。代换常被划分成若干子盒（图 2.4），许多密码算法设计中仅 S 盒是非线性部件，因此 S 盒对整个密码算法的安全强度有重要影响。

随着分组密码方案的不断丰富，对分组密码安全性的讨论也越来越多。针对分组密码安全性的讨论主要包括差分分析、线性分析、穷举搜索等方面。从理论上讲，差分密码分析和线性密码分析是目前攻击分组密码的最有效方法；而从实际上说，穷举搜索等强力攻击是攻击分组密码的最可靠方法。

截至现在，已有大量文献对分组密码的设计和测试进行研究，并归纳出许多有价值的设计和安全性准则。在设计分组密码时，研究人员应该充分考虑这些准则。在此不予详述，有兴趣

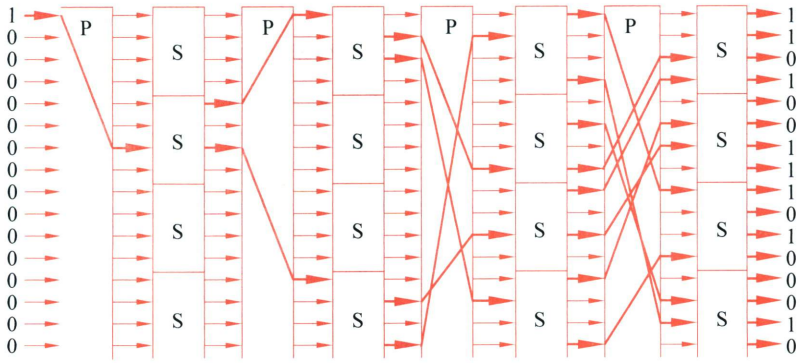

图 2.4　代换和置换网络示意图

的读者可查阅有关的文献。

2.2.1.2　典型算法——数据加密标准 DES

1973 年 5 月，美国国家标准局（现在是美国国家标准与技术研究院，National Institute of Standards and Technology，NIST）公开征集密码体制，这一举措使数据加密标准（DES）出现。DES 是由美国 IBM 公司研制的，是早期的 Lucifer 密码的发展与修改。DES 在 1975 年 3 月 17 日首次被公布，1977 年 1 月 15 日被正式批准并作为美国联邦信息处理标准 FIPS—46，同年 7 月开始生效。当时规定每隔 5 年由美国国家安全局做出评估，并重新批准它是否继续作为联邦加密标准。DES 的最后一次评估在 2001 年 1 月。2002 年 11 月，美国公布了旨在取代 DES 的高级加密标准（AES）算法。

DES 加密算法是第一个广泛用于商用数据保密的密码算法，其分组长度为 64 位，密钥为 56 位（密钥长度为 64 位，其中有 8 位奇偶校验位）。

1. DES 加密过程

DES 加密算法的结构流程如图 2.5 所示。DES 首先利用初始置换对明文进行换位处理，然后进行 16 轮迭代运算，通过每轮的代换和置换操作，达到混淆和扩散的效果，最后通过初始置换的逆置换获得密文。

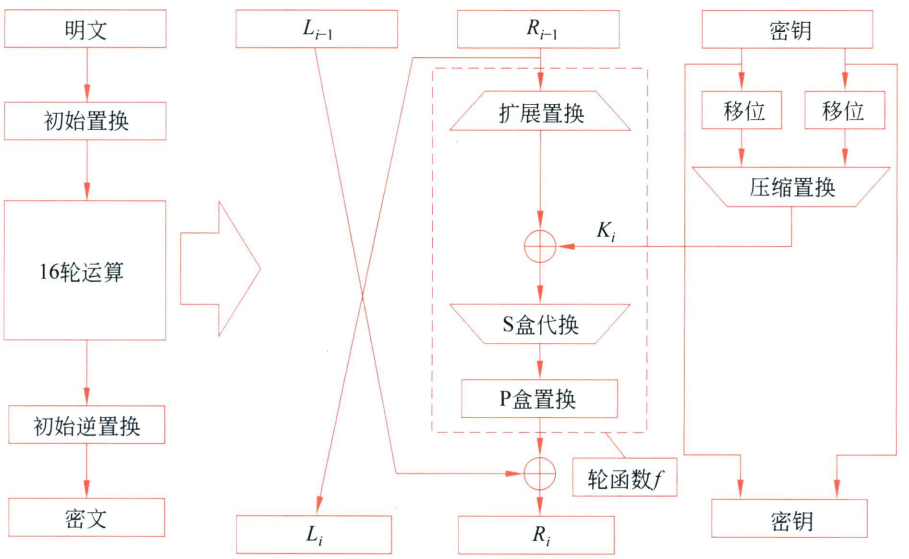

图 2.5　DES 加密算法的结构流程

设 $x=(x_1,x_2,\cdots,x_{64})$是一组待加密的明文块，其中 $x_i\in\{0,1\}$，$1\leqslant i\leqslant 64$。

给定明文 x，通过一个固定的初始置换 IP(表 2.2)来重排输入明文块 x 中的位，得到比特串 $x'=\mathrm{IP}(x)=L_0R_0$，这里 L_0 和 R_0 分别是 x'的前 32 位和后 32 位。初始置换 IP 用于对明文 x 中的各位进行换位，目的在于打乱明文 x 中各位的次序。经过初始置换后，x 变为 $x'=x'_1x'_2\cdots x'_{64}=x_{58}x_{50}\cdots x_7$，即 x 中的第 58 位变为 x'中的第 1 位，以此类推。

表 2.2 初始置换 IP 与初始逆置换 IP^{-1}

初始置换 IP								初始逆置换 IP^{-1}							
58	50	42	34	26	18	10	2	40	8	48	16	56	24	64	32
60	52	44	36	28	20	12	4	39	7	47	15	55	23	63	31
62	54	46	38	30	22	14	6	38	6	46	14	54	22	62	30
64	56	48	40	32	24	16	8	37	5	45	13	53	21	61	29
57	49	41	33	25	17	9	1	36	4	44	12	52	20	60	28
59	51	43	35	27	19	11	3	35	3	43	11	51	19	59	27
61	53	45	37	29	21	13	5	34	2	42	10	50	18	58	26
63	55	47	39	31	23	15	7	33	1	41	9	49	17	57	25

设 $k=(k_1,k_2,\cdots,k_{64})$，其中 $k_i\in\{0,1\}$，$1\leqslant i\leqslant 64$。DES 中与密钥 k 有关的 16 轮迭代可以形式化地表示为

$$\begin{cases}L_i=R_{i-1},\\ R_i=L_{i-1}\oplus f(R_{i-1},K_i),\end{cases}\quad i=1,2,\cdots,16$$

式中：L_i 和 R_i 的长度都是 32 位；$L_0=x'_1x'_2\cdots x'_{32}$；$R_0=x'_{33}x'_{34}\cdots x'_{64}$；$f$ 是一个轮函数；K_i 是由密钥 k 产生的一个 48 位的子密钥。

将 $R_{16}L_{16}$进行初始置换 IP 的逆置换处理后就得到密文 $y=(y_1,y_2,\cdots,y_{64})$。$R_{16}L_{16}$表示将 L_{16}排在 R_{16}的右边。不将 R_{16}与 L_{16}左右交换而直接对 $R_{16}L_{16}$进行逆初始置换处理的目的是使加密和解密可以使用同一算法。

2. 轮函数 f

轮函数 f 是 DES 的核心，其计算过程如图 2.6 所示。

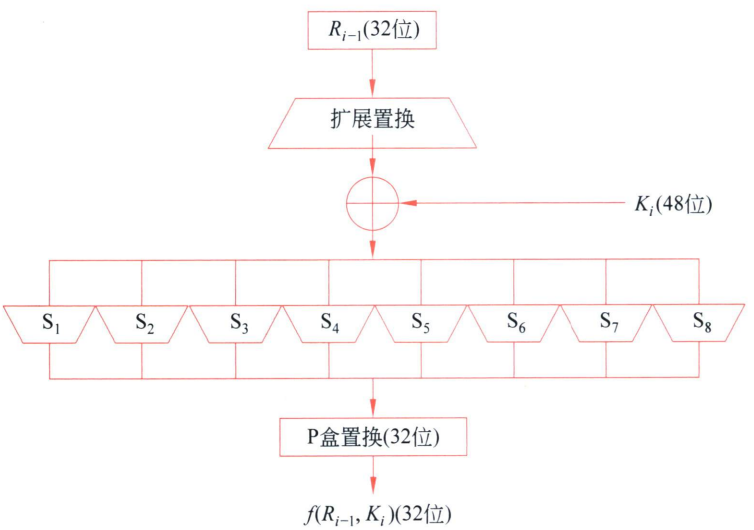

图 2.6 轮函数 f 的计算过程

在每轮计算中，扩展置换(表 2.3)用于先将 32 位的输入扩展为 48 位，然后与子密钥进行按位模 2 加运算，把运算结果从左到右分为 8 组(每组 6 位)，分别输入 8 个 S 盒中(表 2.4)，再用 P 盒置换(表 2.3)对 S 盒代换后的输出进行换位处理后就得到 $f(R_{i-1},K_i)$。

表 2.3 扩展置换和 P 盒置换

扩展置换						P 盒置换			
32	01	02	03	04	05	16	7	20	21
04	05	06	07	08	09	29	12	28	17
08	09	10	11	12	13	1	15	23	26
12	13	14	15	16	17	5	18	31	10
16	17	18	19	20	21	2	8	24	14
20	21	22	23	24	25	32	27	3	9
24	25	26	27	28	29	19	13	30	6
28	29	30	31	32	01	22	11	4	25

表 2.4 轮函数 f 中使用的 8 个 S 盒

S_1															
14	4	13	1	2	15	11	8	3	10	6	12	5	9	0	7
0	15	7	4	15	2	13	1	10	6	12	11	9	5	3	8
4	1	14	8	13	6	2	11	15	12	9	7	3	10	5	0
15	12	8	2	4	9	1	7	5	11	3	14	10	0	6	13
S_2															
15	1	8	14	6	11	3	4	9	7	2	13	12	0	5	10
3	13	4	7	15	2	8	14	12	0	1	10	6	9	11	5
0	14	7	11	10	4	13	1	5	8	12	6	9	3	2	15
13	8	10	1	3	15	4	2	11	6	7	12	0	5	14	9
S_3															
10	0	9	14	6	3	15	5	1	13	12	7	11	4	2	8
13	7	0	9	3	4	6	10	2	8	5	14	12	11	15	1
13	6	4	9	8	15	3	0	11	1	2	12	5	10	14	7
1	10	13	0	6	9	8	7	4	15	14	3	11	5	2	12
S_4															
7	13	14	3	0	6	9	10	1	2	8	5	11	12	4	15
12	8	11	5	6	15	0	3	4	7	2	12	1	10	14	9
10	6	9	0	12	11	7	13	15	1	3	14	5	2	8	4
3	15	0	6	10	1	13	8	9	4	5	11	12	7	2	14
S_5															
2	12	4	1	7	10	11	6	8	5	3	15	13	0	14	9
14	11	2	12	4	7	13	1	5	0	15	10	3	9	8	6

续表

4	2	1	11	10	13	7	8	15	9	12	5	6	3	0	14
11	8	12	7	1	14	2	13	6	15	0	9	10	4	5	3
S_6															
12	1	10	15	9	2	6	8	0	13	3	4	14	7	5	11
10	15	4	2	7	12	9	5	6	1	13	14	0	11	3	8
9	14	15	5	2	8	12	3	7	0	4	10	1	13	11	6
4	3	2	12	9	5	15	10	11	14	1	7	6	0	8	13
S_7															
4	11	2	14	15	0	8	13	3	12	9	7	5	10	6	1
13	0	11	7	4	9	1	10	14	3	5	12	2	15	8	6
1	4	11	13	12	3	7	14	10	15	6	8	0	5	9	2
6	11	13	8	1	4	10	7	9	5	0	15	14	2	3	12
S_8															
13	2	8	4	6	15	11	1	10	9	3	14	5	0	12	7
1	15	13	8	10	3	7	4	12	5	6	11	0	14	9	2
7	11	4	1	9	12	14	2	0	6	10	13	15	3	5	8
2	1	14	7	4	10	8	13	15	12	9	0	3	5	6	11

3. 子密钥生成

在DES算法的16轮迭代中，每轮都需要一个子密钥 K_i 参与。从密钥 k 生成子密钥 K_i 的算法如图2.7所示，密钥 k 中有8位是奇偶校验位，用于检查密钥 k 在产生、分配及存储过程中可能发生的错误。

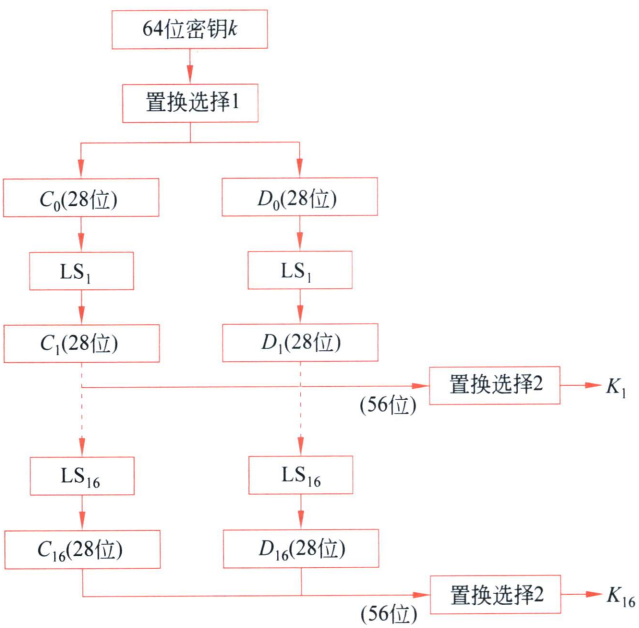

图2.7 子密钥生成算法结构图

置换选择1(表2.5)用于去掉密钥 k 中的校验位,并对其余56位打乱重新排列。置换选择1的输出中前28位作为 C_0,后28位作为 D_0。对于 $1\leqslant i\leqslant 16$,有

$$\begin{cases} C_i = \mathrm{LS}_i(C_{i-1}) \\ D_i = \mathrm{LS}_i(D_{i-1}) \end{cases}$$

式中:LS_i 表示对 C_{i-1} 或 D_{i-1} 进行循环左移变换。当 $i=1,2,9,16$ 时,LS_i 是循环左移1位,其余的 LS_i 是循环左移2位变换。C_iD_i 的长度为56位,置换选择2用于从 C_iD_i 中选取48位作为子密钥 K_i。置换选择2如表2.5所示。

表2.5 置换选择1和置换选择2

置换选择1							置换选择2					
57	49	41	33	25	17	9	14	17	11	24	1	5
1	58	50	42	34	26	18	3	28	15	6	21	10
10	2	59	51	43	35	27	23	19	12	4	26	8
19	11	3	60	52	44	36	16	7	27	20	13	2
63	55	47	39	31	23	15	41	52	31	37	47	55
7	62	54	46	38	30	22	30	40	51	45	33	48
14	6	61	53	45	37	29	44	49	39	56	34	53
21	13	5	28	20	12	4	46	42	50	36	29	32

4. DES解密过程

DES算法是对称的,既可用于加密又可用于解密。只不过在16次迭代中使用的子密钥的次序正好相反。解密时,第一次迭代使用子密钥 K_{16},以此类推。解密过程的16次迭代可以形式化表示为

$$\begin{cases} R_{i-1} = L_i, \\ L_{i-1} = R_i \oplus f(L_i, K_i), \end{cases} \quad i = 16,15,\cdots,1$$

5. DES的安全性

在DES中,初始置换IP和逆初始置换 IP^{-1} 各使用一次,使用这两个置换的目的是把数据彻底打乱重新排列。它们对数据加密所起的作用不大,因为它们与密钥无关且置换关系固定,所以一旦公开,它们对数据的加密便无多大价值。

由前面的算法介绍不难看出,在DES算法加密过程中除了S盒是非线性变换外,其余变换均为线性变换。因此,S盒是DES算法安全的关键。任意改变S盒输入中的一位,其输出至少有两位发生变化。由于在DES中使用了16次迭代,所以即使改变明文或密钥中的1位,密文中都会大约有32位发生变化。S盒的设计原则一直没有完全公开,人们怀疑S盒的设计中可能隐藏着某种"陷门",它可以使了解陷门的人能够成功地进行密码分析。经过多年来的研究,人们的确发现了S盒的许多规律,但至今还没有发现S盒的致命缺陷。

由于DES算法是公开的,因此其安全性完全依赖于所用的密钥。在算法使用过程中,每次迭代时都有一个子密钥供加密使用。子密钥的产生也很有特色,它确保密钥中各位的使用次数基本相等。实验表明,56位密钥中每位的使用次数为12~15次。在实际使用中,需要注意的是DES算法存在一些弱密钥。弱密钥指一个密钥产生的所有子密钥都是相同的,此时对消息加密两次就可以恢复出明文。虽然DES算法有弱密钥现象,但是弱密钥所占比例很小,

可以在选取密钥时避开使用，因此对其安全性影响不大。

随着密码分析技术和计算能力的提高，DES的安全性受到质疑和威胁。密钥长度较短是DES的一个主要缺陷。DES的实际密钥长度为56位，密钥量仅为$2^{56}\approx10^{17}$，就目前计算设备的计算能力而言，DES不能抵抗对密钥的穷举分析攻击。1998年7月，电子前沿基金会(Electronic Frontier Foundation，EFF)使用一台价值25万美元的计算机在56小时内成功地破译了DES。在1999年1月，电子前沿基金会仅用22小时15分就成功地破译了DES。

DES的密钥长度被证明不能满足安全需求，为了提高DES的安全性能，并充分利用有关DES的软件和硬件资源，人们提出一种简单的改进方案——多重DES。多重DES就是使用多个密钥利用DES对明文进行多次加密。例如，三重DES可将密钥长度增加到112位或168位，可以提高抵抗对密钥穷举搜索攻击的能力。除密钥长度因素外，DES加密算法还有一些其他缺陷，如在软件环境下实现效率较低。尽管如此，作为迄今为止世界上最为广泛使用和流行的一种分组密码算法，DES对于推动密码理论的发展和应用起到了重大的作用，对于掌握分组密码的基本理论、设计思想和实际应用仍然有着重要的参考价值。

2.2.1.3 国密算法——SM4分组密码算法

SM4密码算法是中国国家密码管理权威机构发布的国内第一个面向无线局域网产品使用的商用分组密码算法。SM4于2012年3月列入国家密码行业标准《SM4分组密码算法》(GM/T 0002—2012)，2016年8月被列入国家标准《信息安全技术 SM4分组密码算法》(GB/T 32907—2016)，2021年正式成为ISO/IEC国际标准。该算法的分组长度为128位，密钥长度为128位。加密算法与密钥扩展算法都采用32轮非线性迭代结构，解密算法与加密算法的结构相同，只是轮密钥的使用顺序相反，解密轮密钥是加密轮密钥的逆序。

1. SM4加解密过程

SM4算法采用非线性迭代结构，以字为单位进行加密运算，以下介绍中使用Z_2^e表示e位的向量集。Z_2^{32}中的元素称为字，Z_2^8中的元素称为字节，用符号“⊕”表示32位按位异或，定义反序变换R为$R(A_0,A_1,A_2,A_3)=(A_3,A_2,A_1,A_0)$，其中$A_i\in Z_2^{32}(i=0,1,2,3)$，用符号“$i<<<$”表示32位循环左移$i$位。

设输入的128位明文为(X_0,X_1,X_2,X_3)，其中$X_i\in Z_2^{32}(i=0,1,2,3)$。密文输出为$(Y_0,Y_1,Y_2,Y_3)$，其中$Y_i\in Z_2^{32}(i=0,1,2,3)$，轮密钥为$rk_i\in Z_2^{32}(i=0,1,\cdots,31)$。

SM4算法的加密变换为

$$\begin{aligned}X_{i+4}&=F(X_i,X_{i+1},X_{i+2},X_{i+3},rk_i)\\&=X_i\oplus T(X_{i+1}\oplus X_{i+2}\oplus X_{i+3}\oplus rk_i)(i=0,1,\cdots,31)\end{aligned}$$

SM4算法一轮的加密流程如图2.8所示。

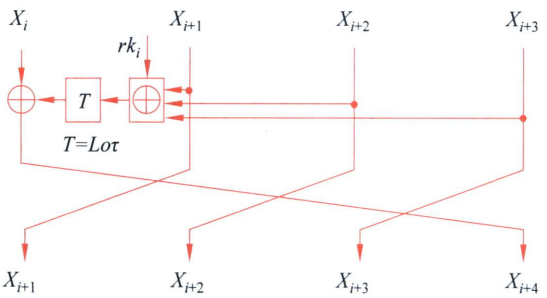

图2.8 SM4算法一轮的加密流程

经过 32 轮迭代输出$(X_{32},X_{33},X_{34},X_{35})$，经过反序变换得到密文

$$(Y_0,Y_1,Y_2,Y_3)=R(X_{32},X_{33},X_{34},X_{35})=(X_{35},X_{34},X_{33},X_{32})$$

SM4 算法的解密变换与加密变换结构相同，不同的仅是轮密钥的使用顺序。

2. 轮函数 F

轮函数 F 定义为 $F(X_i,X_{i+1},X_{i+2},X_{i+3},rk_i)=X_i\oplus T(X_{i+1}\oplus X_{i+2}\oplus X_{i+3}\oplus rk_i)$，下面介绍轮函数 F 中包含的合成变换 T。$T:Z_2^{32}\to Z_2^{32}$ 是一个可逆变换，由非线性变换 τ 和线性变换 L 复合而成，即 $T(\cdot)=L\circ\tau(\cdot)=L(\tau(\cdot))$。这里，非线性变换 τ 由 4 个并行的 S 盒构成。S 盒为固定的 8 位输入、8 位输出的置换，记为 Sbox(·)。

设输入为 $A=(a_0,a_1,a_2,a_3)$，输出为 $B=(b_0,b_1,b_2,b_3)$，其中 $a_i,b_i\in Z_2^8(i=0,1,2,3)$，则

$$(b_0,b_1,b_2,b_3)=\tau(A)=(\text{Sbox}(a_0),\text{Sbox}(a_1),\text{Sbox}(a_2),\text{Sbox}(a_3))$$

式中：S 盒中数据如表 2.6 所示。S 盒中的数据均采用十六进制数表示，设 S 盒输入“ef”，则经 S 盒后的值为表 2.6 中第 e 行第 f 列的值，即 Sbox(ef)=0x84。

表 2.6 SM4 算法的 S 盒

	0	1	2	3	4	5	6	7	8	9	a	b	c	d	e	f
0	d6	90	e9	fe	cc	e1	3d	b7	16	b6	14	c2	28	fb	2c	05
1	2b	67	9a	76	2a	be	04	c3	aa	44	13	26	49	86	06	99
2	9c	42	50	f4	91	ef	98	7a	33	54	0b	43	ed	cf	ac	62
3	e4	b3	1c	a9	c9	08	e8	95	80	df	94	fa	75	8f	3f	a6
4	47	07	a7	fc	f3	73	17	ba	83	59	3c	19	e6	85	4f	a8
5	68	6b	81	b2	71	64	da	8b	f8	eb	0f	4b	70	56	9d	35
6	1e	24	0e	5e	63	58	d1	a2	25	22	7c	3b	01	21	78	87
7	d4	00	46	57	9f	d3	27	52	4c	36	02	e7	a0	c4	c8	9e
8	ea	bf	8a	d2	40	c7	38	b5	a3	f7	f2	ce	f9	61	15	a1
9	e0	ae	5d	a4	9b	34	1a	55	ad	93	32	30	f5	8c	b1	e3
a	1d	f6	e2	2e	82	66	ca	60	c0	29	23	ab	0d	53	4e	6f
b	d5	db	37	45	de	fd	8e	2f	03	ff	6a	72	6d	6c	5b	51
c	8d	1b	af	92	bb	dd	bc	7f	11	d9	5c	41	1f	10	5a	d8
d	0a	c1	31	88	a5	cd	7b	bd	2d	74	d0	12	b8	e5	b4	b0
e	89	69	97	4a	0c	96	77	7e	65	b9	f1	09	c5	6e	c6	84
f	18	f0	7d	ec	3a	dc	4d	20	79	ee	5f	3e	d7	cb	39	48

非线性变换 τ 的输出作为线性变换 L 的输入。设输入为 $B\in Z_2^{32}$，输出为 $C\in Z_2^{32}$，则

$$C=L(B)=B\oplus(B<<<2)\oplus(B<<<10)\oplus(B<<<18)\oplus(B<<<24)$$

3. 子密钥生成

SM4 算法轮密钥由加密密钥通过密钥扩展算法生成。设加密密钥 $MK=(MK_0,MK_1,MK_2,MK_3)$，其中 $MK_i\in Z_2^{32}(i=0,1,2,3)$。令 $K_i\in Z_2^{32}(i=0,1,\cdots,35)$，轮密钥 $rk_i\in Z_2^{32}(i=0,1,\cdots,31)$ 生成方法为

$$(K_0,K_1,K_2,K_3)=(MK_0\oplus FK_0,MK_1\oplus FK_1,MK_2\oplus FK_2,MK_3\oplus FK_3)$$

$$rk_i=K_{i+4}=K_i\oplus T'(K_{i+1}\oplus K_{i+2}\oplus K_{i+3}\oplus CK_i)$$

式中：

（1）T'变换与加密算法轮函数中的T基本相同，只是将线性变换L修改为L'：

$$L'(B)=B\oplus(B<<<13)\oplus(B<<<23)$$

（2）系统参数FK的取值，采用十六进制表示为

$FK_0=$(A3B1BAC6)，$FK_1=$(56AA3350)，$FK_2=$(677D9197)，$FK_3=$(B27022DC)。

（3）固定参数CK的取值方法为：设$ck_{i,j}\in Z_2^8$为CK_i的第j字节($i=0,1,\cdots,31;j=0,1,2,3$)，即

$$CK_i=(ck_{i,0},ck_{i,0},ck_{i,0},ck_{i,0})$$

则$ck_{i,j}=(4i+j)\times 7(\mathrm{mod}\ 256)$，由此式可得出32个固定参数$CK_i$，其十六进制表示如表2.7所示。

表2.7 CK_i的取值

00070e15	1c232a31	383f464d	545b6269
70777e85	8c939aa1	a8afb6bd	c4cbd2d9
e0e7eef5	fc030a11	181f262d	343b4249
50575e65	6c737a81	888f969d	a4abb2b9
c0c7ced5	dce3eaf1	f8ff060d	141b2229
30373e45	4c535a61	686f767d	848b9299
a0a7aeb5	bcc3cad1	d8dfe6ed	f4fb0209
10171e25	2c333a41	484f565d	646b7279

2.2.1.4 分组密码工作模式

分组密码是将消息作为数据分组来加密或解密的，而实际应用中大多数消息的长度是不定的，数据格式也不同。当消息长度大于分组长度时，需要将其分成几个分组分别进行处理。为了能灵活地运用基本的分组密码算法，人们设计了不同的处理方式，其被称为分组密码的工作模式，也被称为分组密码算法的运行模式。

下面介绍四个常用的工作模式，即电子编码本(Electronic Code Book，ECB)模式、密码分组链接(Cipher Block Chaining，CBC)模式、输出反馈(Output Feed Back，OFB)模式、密码反馈(Cipher Feed Back，CFB)模式。

1. 电子编码本模式

分组密码在ECB模式工作下的加密、解密过程，如图2.9所示，首先将明文消息分成n个m位组，如果明文长度不是m的整数倍，则在明文末尾填充适当数目的规定符号，使长度为m位的整数倍。对每个明文组用给定的密钥分别进行加密，生成n个相应的密文组。解密和加密的工作模式基本一致。

ECB模式是最容易的运行模式，每个明文分组可以被独立地运行加密，因此可以并行实现。在误差传播方面，单个密文分组中有一个或多个位错误只会影响该分组的解密结果，错误传播较小。但这种模式下，相同明文(在相同密钥下)会得出相同的密文，容易实现统计分析

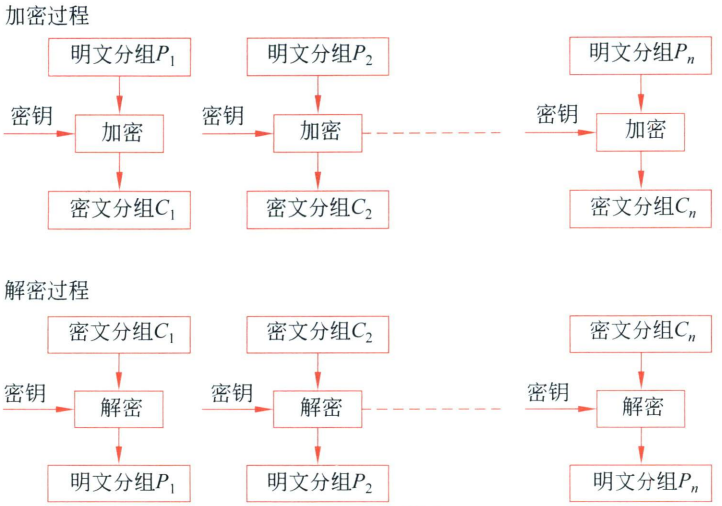

图 2.9 ECB 模式

攻击。

2. 密码分组链接模式

在 CBC 模式下的加密、解密过程，如图 2.10 所示，每个明文组在加密前与前一组密文按位异或运算后，再进行加密变换，首个明文组与一个初始向量 ***IV*** 异或运算。采用 CBC 方式加密，要求收发双方共享加密密钥和初始向量 ***IV***。解密时每组密文先进行解密，再与前组密文进行异或运算，还原出该组明文。

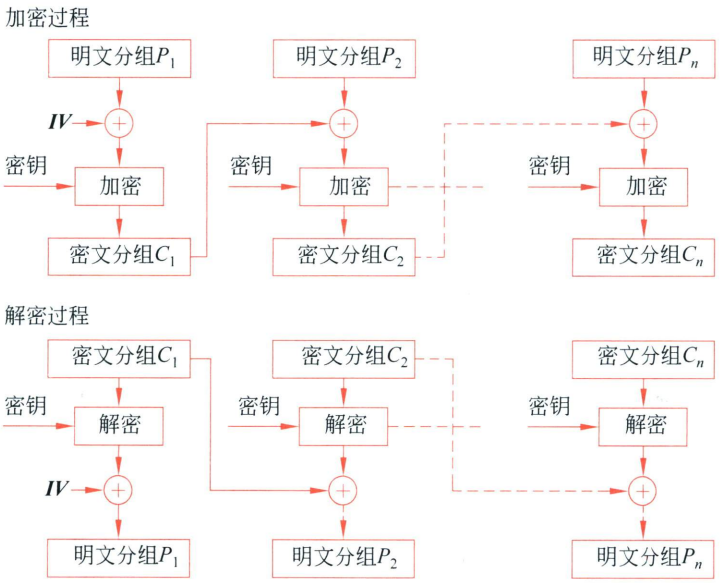

图 2.10 CBC 模式

使用 CBC 模式时，初始化向量 ***IV*** 同密钥一样需要保密。由于引入的反馈机制，因而每个密文分组不仅依赖于产生它的明文分组，还依赖于它前面的所有分组，不能进行并行处理。相同的明文，即使在相同的密钥下也会得到不同的密文分组，隐藏了明文的统计特性。同时，密文分组中的一个位错误会影响到本组和其后一个分组的解密，错误传播为两组。

3. 密码反馈模式

在CFB模式下，可以利用分组密码实现实时的流操作。将发送的字符流中任何一个字符用面向字符的工作模式加密后立即发送，其原理如图2.11所示，其中传输单元（移位寄存器）是s位，一般$s=8$。此时，明文被分成s位的片段而不是使用的基本分组密码的分组长度。使用CFB模式时，任意明文单元的密文都是前面所有明文的函数。

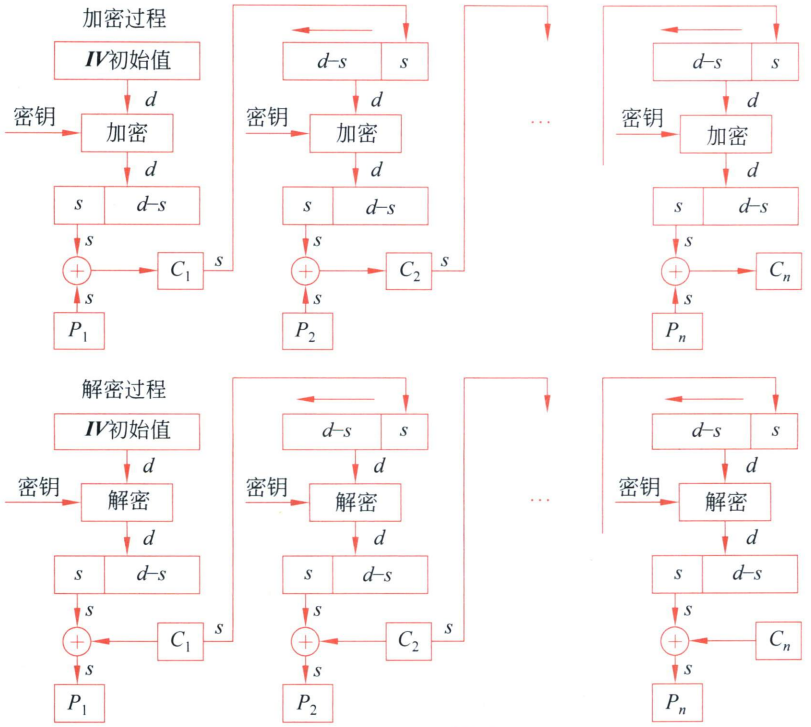

图2.11 CFB模式

加密时，设加密算法的输入是d位移位寄存器，其初值为某个初始向量***IV***。加密算法输出的最左（最高有效位）s位与明文的第一个单元P_1进行异或，产生出密文的第1个单元C_1。传送该单元并将输入寄存器的内容左移s位，用C_1补齐最右边（最低有效位）s位。这一过程持续到明文的所有单元都被加密为止。解密时，将加密算法输出的最左（最高有效位）s位与密文的相应单元异或产生明文，反馈到输入寄存器的值为密文单元。注意：在数据解密过程中使用的是指定分组密码的加密算法而不是解密算法。

在CFB模式中，需要额外的初始向量，消息被看作位流，无须分组填充，无须整个数据分组在接收完后才能进行加解密。所有加密都使用同一密钥，密文块需按顺序逐一解密。另外，CFB模式中数据加解密的速率降低，其数据率不会太高，同时对信道错误较敏感且会造成错误传播。

4. 输出反馈模式

OFB模式与CFB模式相似，不同之处在于OFB模式将前一次加密算法输出的s位反馈送入移位寄存器的最右边（图2.12），而CFB模式是将密文单元反馈到移位寄存器中。因为OFB模式的反馈机制独立于明文和密文，这种方法也被称为“内部反馈”。

在OFB模式中，初始向量***IV***无须保密，但各条消息必须选用不同的***IV***；密钥相同时，明文中相同的组产生不相同的密文块。CFB模式和OFB模式都是将消息看作位流，无须分组

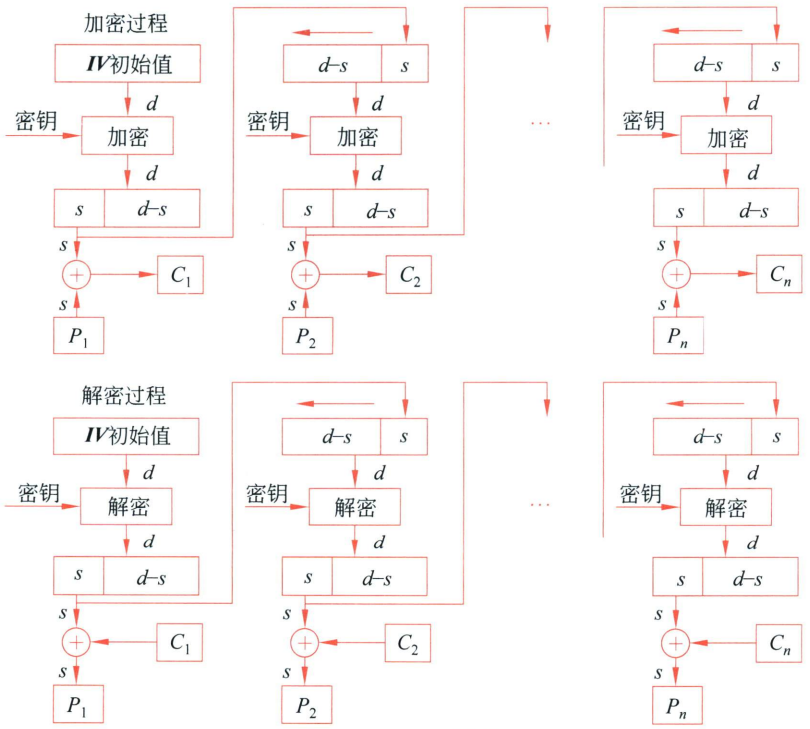

图 2.12 OFB 模式

填充。OFB 模式是 CFB 模式的一种改进，不存在位错误传播；密钥流可以在已知消息之前计算，不需要按顺序解密。但是，OFB 模式比 CFB 模式更易受到对消息流的篡改攻击。比如，在密文中取 1 位的补，那么在恢复的明文中相应位置的位也为原位的补。因此使得敌手有可能通过同时对消息校验部分的篡改和对数据部分的篡改，以纠错码不能检测的方式篡改密文。

密码工作模式通常是基本密码模块、反馈和一些简单运算的组合，应当力求简单、有效和易于实现。适当选用分组模式能够为密文组提供一些其他的性质，如隐藏明文的统计特性和数据格式、控制错误传播等，以提高整体的安全性，降低删除、重放、插入和伪造等攻击的机会。

2.2.2 序列密码

序列密码（也被称为流密码）是一类重要的密码体制，也是手工和机械密码时代的主流密码。序列密码通常被认为起源于一次一密密码体制，由于随机密钥序列的产生、存储及分配等方面存在一定的困难，一次一密体制在当时并没有得到广泛的应用。在 20 世纪 50 年代，由于数字电子技术的发展，使密钥序列可以方便地利用以移位寄存器为基础的电路来产生，从而促使线性和非线性移位寄存器理论迅速发展，再加上有效的数学工具（如代数和谱分析理论）的引入，使得序列密码理论迅速发展，并逐步走向成熟阶段。同时由于具有实现简单、速度快，以及错误传播少的优点，使序列密码在实际应用中，特别是在专用和机密机构中仍保持优势。

序列密码属于对称密码体制，与分组密码相比较：分组密码把明文分成相对比较大的块，对于每块使用相同的加密函数进行处理，分组密码是无记忆的；序列密码处理的明文长度为 1 位，而且序列密码是有记忆的。序列密码又被称为状态密码，因为它的加密不仅与密钥和明文有关系，还和当前状态有关。两者区别不是绝对的，若把分组密码增加少量的记忆模块就形成

了一种序列密码。

序列密码通常划分为同步序列密码和自同步序列密码两大类。如果密钥序列的产生独立于明文消息，则此类序列密码为同步序列密码。在同步序列密码中，密（明）文符号是独立的，一个错误传输只会影响一个符号，不影响后面的符号。但其缺点是：一旦接收端和发送端的种子密钥和内部状态不同步，解密就会失败，两者必须立即借助外界手段重新建立同步。如果密钥序列的产生是密钥及固定大小的以往密文位的函数，则这种序列密码被称为自同步序列密码或非同步序列密码。自同步序列密码的优点是即使接收端和发送端不同步，只要接收端能连续地正确接收到 n 个密文符号，就能重新建立同步。因此自同步序列密码具有有限的差错传播，且较同步序列密码的分析困难得多。

1. 序列密码基本原理

序列密码将明文划分成字符（如单个字母）或其编码的基本单元（如0、1数字），字符分别与密钥序列作用进行加密，解密时以同步产生的同样的密钥序列实现，其基本流程如图2.13所示。保持收发两端密钥序列的精确同步是实现可靠解密的前提。

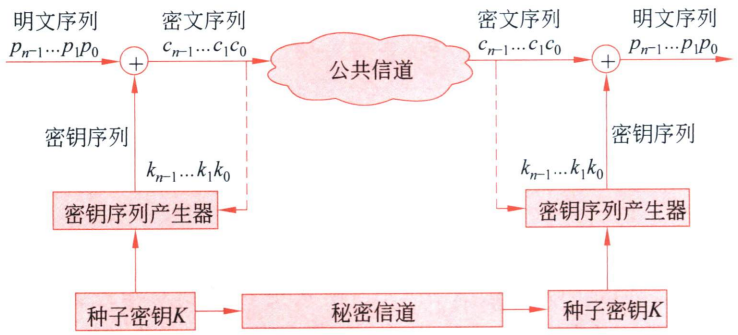

图 2.13 序列密码流程示意图

种子密钥 K 控制密钥序列产生器，产生密钥序列$\{k_i\}$，$i \geqslant 0$。明文序列 $m=m_1m_2\cdots m_i\cdots$ $(m_i \in M)$与密钥位模2加，产生密文序列 $c=c_1c_2\cdots c_i\cdots$，其中：$c_i=E(k_i,m_i)=m_i \oplus k_i$。若密钥序列是一个完全随机的非周期序列，则可以实现一次一密体制。序列密码的安全强度主要依赖密钥序列的随机性，因此设计一个好的密钥序列产生器，使其产生随机的密钥序列是序列密码体制的关键。

密钥序列生成器的内部可将其分成两部分——驱动器部分和非线性组合器部分（图2.14），其中驱动器部分产生控制生成器的状态序列，并控制生成器的周期和统计特性。驱动器一般利用线性反馈移位寄存器（Linear Feedback Shift Register，LFSR），特别是利用最长周期或 m 序列产生器实现。非线性组合器部分对驱动器部分的各个输出序列进行非线性组合，控制和提高产生器输出序列的统计特性、线性复杂度和不可预测性等，从而保证输出密钥序列的安全强度。

密钥序列生成器的设计基本要求如下。

（1）种子密钥 K 的长度足够大，一般应在128位以上。

（2）密钥序列生成器生成的密钥序列$\{k_i\}$具有极大周期。

（3）密钥序列$\{k_i\}$具有均匀的 n-元分布，即在一个周期环上，某特定形式的 n-长位串与其求反，两者出现的频数大抵相当。

（4）利用统计方法由密钥序列$\{k_i\}$提取关于种子密钥 K 的信息在计算上不可行。

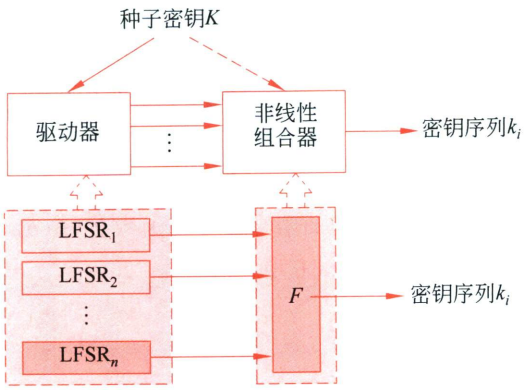

图 2.14 密钥序列生成器组成

(5) 种子密钥 K 任一位的改变要引起密钥序列$\{k_i\}$在全貌上的变化。

(6) 密钥序列$\{k_i\}$不可预测,密文及相应明文的部分信息不能确定整个密钥序列。

为了保证输出密钥序列的安全强度,对组合函数 F 有下列要求。

(1) F 将驱动序列变换为滚动密钥序列,当输入二元随机序列时,输出也为二元随机序列。

(2) 对给定周期的输入序列,构造的 F 使输出序列的周期尽可能大。

(3) 对给定复杂度的输入序列,应构造 F 使输出序列的复杂度尽可能大。

(4) F 的信息泄露应极小化(从输出难以提取有关密钥序列产生器的结构信息)。

(5) F 应易于工程实现,工作速度极高。

(6) 在需要时,F 易于在密钥控制下工作。

序列密码在实时处理方面效率高,具有实现简单、便于硬件实现、加解密处理速度快等特点,在实际应用中有很大的优势,其典型的应用领域包括无线通信、外交通信等。

2. 线性反馈移位寄存器

序列密码的关键是设计一个随机性好的密钥序列生成器,为了研究密钥序列生成器,挪威政府的首席密码学家 Ernst Selmer 于 1965 年提出了移位寄存器理论,它是序列密码中研究随机密钥流的主要数学工具。尤其是线性反馈移位寄存器,因其实现简单、速度快、有较为成熟的理论等优点,成为构造密码流生成器的最重要部件之一。

反馈移位寄存器(Feedback Shift Register,FSR)是由 n 位的寄存器和反馈函数组成,如图 2.15 所示,n 位寄存器中的初始值称为移位寄存器的初态。

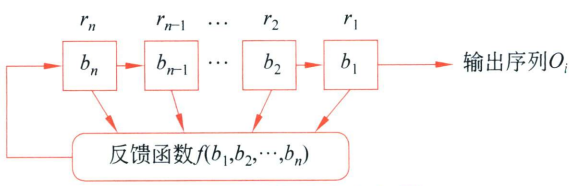

图 2.15 反馈移位寄存器

工作原理:移位寄存器中所有位的值右移 1 位,最右边的一个寄存器移出的值是输出位,最左边一个寄存器的值由反馈函数的输出值填充,此过程称为进动 1 拍。反馈函数 f 是 n 个变元$(b_1,b_2,\cdots,b_n)$的布尔函数。移位寄存器根据需要不断地进动 m 拍,便有 m 位的输出,形成输出序列 $O_1,O_2,\cdots,O_m$。

线性反馈移位寄存器(LFSR)是一种特殊的 FSR,其反馈函数是线性函数,即为移位寄存

器中某些位的异或，参与运算的这些位称为抽头位。

一个 n 阶 LFSR 的有效状态为 2^n-1（全 0 状态除外，因全 0 状态的输出序列一直为全 0），也即理论上能够产生周期为 2^n-1 的伪随机序列。线性反馈移位寄存器输出序列的性质完全由其反馈函数决定。选择合适的反馈函数便可使序列的周期达到最大值 2^n-1，周期达到最大值的序列称为 *m* 序列。

【例 2-1】 一个 3 阶的线性反馈移位寄存器，反馈函数为 $f(b_1,b_2,b_3)=b_1\oplus b_3$，初态为 $(b_1b_2b_3)=100$，输出序列生成过程如图 2.16 所示。

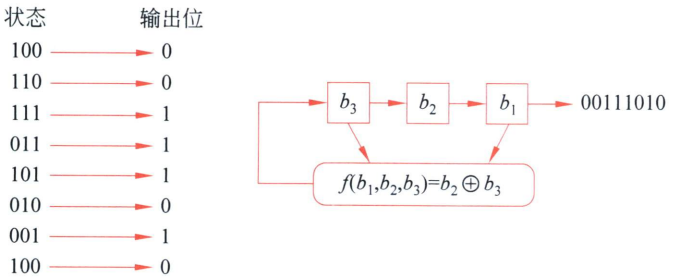

图 2.16 输出序列生成过程举例

此输出序列周期长度为 $7=2^3-1$，因此为 m 序列。

尽管 m 序列的随机性能较好，且在所有同阶线性移位寄存器生成序列中其周期最长，但从序列密码安全性角度来看，m 序列并不适合直接作为密钥序列来使用。因此，密钥序列生成器仅有线性移位寄存器是不够的，还需要非线性组合部分。

一般来说，驱动部分可由 m 序列或其他长周期的 LFSR 序列组成，用于控制密钥流生成器的状态序列，并为非线性组合器部分提供伪随机性质良好的序列；非线性组合器部分利用驱动器部分生成的状态序列生成满足要求的密码特性好的密钥流序列。

密钥序列生成器设计符合 Shannon 的“扩散”和“混淆”两条密码学的基本原则。驱动器部分利用 LFSR 将密钥 k 扩散成周期很大的状态序列，而状态序列与密钥 k 间的关系经非线性组合器混淆后被隐蔽。

【例 2-2】 密钥序列生成器实例——Geffe 生成器，如图 2.17 所示。

Geffe 生成器有两个 LFSR 作为复合器的输入，第三个 LFSR 控制复合器的输出。如果 a_1、a_2 和 a_3 是三个 LFSR 的输出，则 Geffe 生成器的输出表示为

$$b=(a_1\wedge a_2)\oplus(\neg a_1\wedge a_3)=(a_1\wedge a_2)\oplus(a_1\wedge a_3)\oplus a_3。$$

这个生成器的周期是三个 LFSR 周期的最小公倍数，它能实现序列周期的极大化，且 0 和 1 之间的分布大体是平衡的。

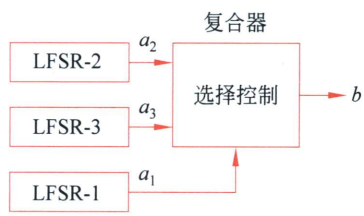

图 2.17 Geffe 生成器示意图

3. 典型算法——A5 算法

A5 算法是 GSM 系统中使用的加密算法之一，主要用于加密手机终端与基站之间传输的语音和数据。一个 GSM 语言消息被转换成一系列的帧，每帧具有 228 位，每帧用 A5 进行加密。

A5 算法是一种典型的基于线性反馈移位寄存器的序列密码算法，构成 A5 加密器主体的 LFSR 有 3 个，组成了一个集互控和停走于一体的钟控模型。线性移位寄存器（A、B、C）的长度各不相同，其中 A 有 19 位，B 有 22 位，C 有 23 位。它们的移位方式都是由低位移向高位。

每次移位后，最低位就要补充一位，补充的值由寄存器中的某些抽头位进行异或运算的结果决定，如运算的结果为1，则补充1，否则补充0。在3个LFSR中，A的抽头位置为18、17、16、13；B的抽头位置为21、20、16、12；C的抽头位置为22、21、18、17。3个LFSR输出的异或值作为A5算法的输出。A5算法的主体部分示意图如图2.18所示。

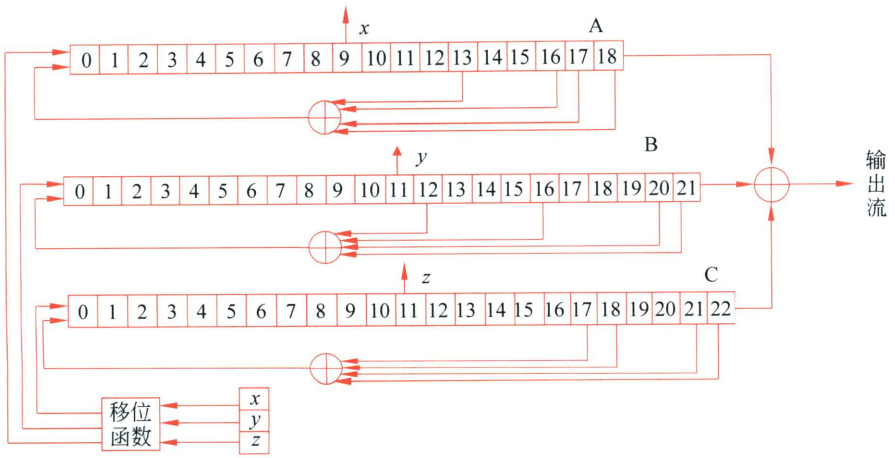

图2.18 A5算法的主体部分示意图

移位是由时钟控制的，且遵循“择多”的原则。即从每个寄存器中取出一个中间位并进行判断，三个数中占多数的寄存器参加移位，其余的不移位。比如，在取出的三个中间位中有两个为1，则为1的寄存器进行一次移位，而为0的不移。反过来，若三个中间位中有两个为0，则为0的寄存器进行一次移位，而为1的不移。这种机制保证了每次至少有2个LFSR被驱动移位。

4. 国密算法——祖冲之序列密码算法

祖冲之序列密码算法(简称ZUC算法)是由我国自主设计的密码算法，包括祖冲之算法、加密算法128-EEA3和完整性算法128-EIA3。2004年3GPP(The 3rd Generation Partner Project)启动LTE(Long Term Evolution)计划，2010年年底LTE被指定为第四代移动通信标准，简称4G通信标准。

2011年我国推荐ZUC算法的128-EEA3加密算法和128-EIA3完整性算法成为3GPP LTE保密性和完整性算法标准，即第四代移动通信加密标准，2012年ZUC算法被发布为国家密码行业标准，2016年被发布为国家标准。祖冲之密码算法是第一个成为国际标准的我国自主研制的密码算法，对我国电子信息产业具有非常重要的意义。算法具体描述可通过登录网址http://gmssl.org/docs/zuc.html进一步了解。

2.3 公钥密码

对称密码体制虽然可以在一定程度上解决保密通信的问题，但随着计算机和网络的飞速发展，保密通信的需求越来越广泛，对称密码体制的局限性也逐渐显露，主要表现如下。

(1) 密钥分配问题。通信双方要进行加密通信，需要通过秘密的安全信道协商加密密钥，而这种安全信道可能很难实现。

(2) 密钥管理问题。在有多个用户的网络中，任何两个用户之间都需要有共享的密钥，当

网络中的用户数很大时，需要管理的密钥数目非常大。

（3）难以实现不可否认功能。当用户 A 收到用户 B 的消息时，无法向第三方证明此消息确实来源于 B，也无法防止事后 B 否认发送过消息。

公钥密码体制（也称为非对称密码体制）为密码学的发展提供了新的理论和技术思想，是现代密码学最重要的发明，也可以说是目前密码学发展史上最伟大的革命。一方面，公钥密码的算法是基于数学函数的，而不是建立在字符或位方式操作上的。另一方面，与对称密码加、解密使用同一密钥不同，公钥密码使用两个独立的密钥，且加密密钥可以公开。这两个密钥的使用对密钥的管理、认证都有重要的意义。本节将介绍公钥密码的基本原理、特点及典型的算法。

2.3.1 公钥密码概述

1976 年，Diffie 和 Hellman 在《密码学的新方向》一文中提出了公钥密码的思想，开创了公钥密码学的新纪元。利用公钥密码体制，通信双方无须事先交换密钥就可以进行保密通信，其通信模型如图 2.19 所示。信息发送前，发送者首先要获取接收者发布的加密密钥，加密时使用该密钥将明文加密成密文，加密密钥也被称为公开密钥，简称公钥；解密时接收者使用解密密钥对密文进行处理，还原明文，解密密钥需要保密，因此也被称为私有密钥，简称私钥。

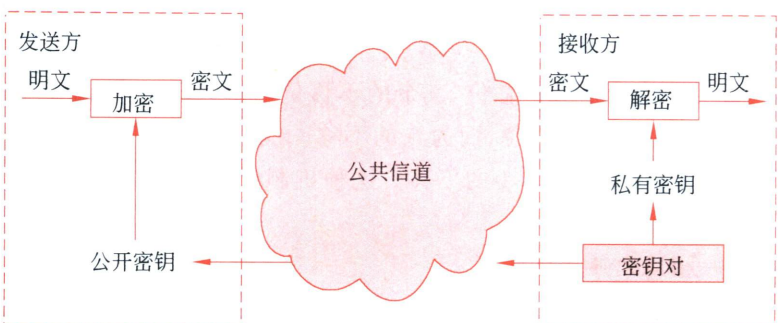

图 2.19 公钥密码体制通信模型

Diffie 和 Hellman 在文章中并没有给出一个具体的公钥密码算法，但首次提出了单向陷门函数的概念，将公钥密码体制的研究归结为单向陷门函数的设计，为公钥密码的研究指明了方向。

如果函数 $f(x)$ 被称为单向陷门函数，必须满足以下三个条件。

（1）给定 x，计算 $y=f(x)$ 是容易的。

（2）给定 y，计算 x 使 $y=f(x)$ 是困难的（计算 $x=f^{1}(y)$ 困难指计算上相当复杂，已无实际意义）。

（3）存在 δ，已知 δ 时，对给定的任何 y，若相应的 x 存在，则计算 x 使 $y=f(x)$ 是容易的。

对于以上条件仅满足前两条的称为单向函数；第三条称为陷门性，δ 称为陷门信息。当用陷门函数 f 作为加密函数时，可将 f 公开，这相当于公开加密密钥 P_k。f 函数的设计者将 δ 保密，用作解密密钥，此时 δ 即为私有密钥 S_k。由于加密函数是公开的，任何人都可以将信息 x 加密成 $y=f(x)$，然后发送给函数的选取者。只有函数选取者拥有 S_k，可以利用 S_k 求解 $x=f^{1}(y)$。单向陷门函数的第二条性质也表明窃听者由截获的密文 $y=f(x)$ 推测 x 是不可行的。

公钥密码体制采用的加密密钥(公钥)和解密密钥(私钥)是不同的。公钥密码提出后,立刻受到了人们的普遍关注。由于加密密钥是公开的,密钥的分配和管理就很简单,而且能够很容易地实现数字签名,因此能够满足电子商务应用的需要。在实际应用中,公钥密码体制并没有完全取代对称密码体制,这是因为公钥密码体制是基于某种数学难题的,计算非常复杂,它的运行速度远比不上对称密码体制。因此,在实际应用中可以利用二者各自的优点,采用对称密码体制加密文件,而采用公钥密码体制加密"加密文件"的密钥,就是混合加密体制。混合加密体制较好地解决了运算速度和密钥分配管理的问题。

2.3.2 典型公钥密码算法

从公钥密码的思想被提出以来,国际上已经出现了多种公钥密码算法。这些算法的安全性主要基于复杂的数学难题。对于某种数学难题,如果利用通用的算法计算出密钥的时间越长,那么基于这一数学难题的公钥密码体制就被认为越安全。根据所基于的数学难题来分类,公钥密码体制可以分为以下三类:基于大整数分解问题的公钥密码体制、基于有限域上离散对数问题的公钥密码体制、基于椭圆曲线离散对数问题的公钥密码体制。下面介绍一下著名的 RSA 公钥密码算法和 SM2 椭圆曲线公钥密码算法。

2.3.2.1 RSA 公钥密码算法

RSA 密码是目前应用最广泛的公钥密码体制,该算法是由美国的 Ron Rivest、Adi Shamir 和 Leonard Adleman 三人于 1978 年提出的。它既能用于加密,又能用于数字签名,易于理解和实现,是第一个安全、实用的公钥密码体制。

1. RSA 算法的数学基础

RSA 算法的基础是欧拉定理,它的安全性依赖于大整数因子分解的困难性。为了方便理解 RSA 密码算法,首先介绍一下欧拉定理和大整数因子分解问题。

欧拉定理:若正整数 a 与 n 互素,则 $a^{\varphi(n)}=1 \bmod n$。欧拉函数 $\varphi(n)$ 是对于一个正整数 n,比 n 小但与 n 互素的正整数的个数。特别地,如果 p 是素数,则 $\varphi(p)=p-1$。如果有两个素数 p 和 q,且 $n=pq$,则 $\varphi(n)=(p-1)(q-1)$。欧拉定理的证明可查阅其他参考资料。

大整数因子分解问题:已知 p 和 q 为两个大素数,则求 $N=pq$ 是容易的;但已知 N 是两个大素数的乘积,要求将 N 分解,则在计算上是困难的,其运行时间程度接近于不可行。实际上,如果一个大的有 n 个二进制数位长度的数是两个差不多大小的素数的乘积,目前还没有很好的算法能在多项式时间内分解它。

算法时间复杂性是衡量算法有效性的常用标准。如果输入规模为 n 时,一个算法的运行时间复杂度为 $O(n)$,称此算法为线性时间的;运行时间复杂度为 $O(n^k)$(其中 k 为常量),称此算法为多项式时间的;若有某常量 t 和多项式 $h(n)$,使算法的运行时间复杂度为 $O(t^{h(n)})$,则称此算法为指数时间的。

一般说来,在线性时间和多项式时间内可以解决的问题被认为是可行的,而任何比多项式时间更坏的,尤其是指数时间可解决的问题被认为是不可行的。需要注意的是,如果输入规模太小,即使很复杂的算法也会变得可行。

2. RSA 算法描述

选取两个不同的大素数 p 和 q,为了获得最大程度的安全性,p 和 q 的长度一样。计算它们的乘积 $n=pq$,令 $\varphi(n)=(p-1)(q-1)$。

随机选取一个整数 e,$1\leqslant e\leqslant\varphi(n)$,$(\varphi(n),e)=1$。因为$(\varphi(n),e)=1$,所以在模 $\varphi(n)$下,

可以计算出满足 $d \cdot e \equiv 1 \bmod \varphi(n)$ 的 d（可利用推广的欧几里得算法求得）。

e 和 n 为公钥，d 是私钥。不再需要两个素数 p 和 q，可以销毁，但决不能泄露。

加密算法：加密消息 m 时，首先将它分成比 n 小的数据分组。对于其中任一个分组 x 计算 $y \equiv x^e \bmod n$。

解密算法：解密消息时，对于任一个密文块 y，计算 $x \equiv y^d \bmod n$。

算法证明：因为 $d \cdot e \equiv 1 \bmod \varphi(n)$，可将 ed 表示为 $ed = k\varphi(n)+1$，其中 k 为任意整数。公式为

$$y^d \bmod n = (x^e)^d \bmod n = x^{ed} \bmod n = x^{k\varphi(n)+1} \bmod n$$

由欧拉定理知 $x^{\varphi(n)} \equiv 1 \bmod n$，有 $x^{k\varphi(n)} \equiv 1 \bmod n$。因此，$x^{k\varphi(n)+1} \bmod n = x \bmod n$，即

$$y^d \bmod n = x$$

解密的结果是得到明文 x。

下面举一个简单的例子说明这一过程。

【例 2-3】 取两个素数 $p=7$，$q=17$，计算出 $n=pq=7\times17=119$，于是得到 $\varphi(n)=96$。选择一个与 96 互素的正整数 e，假设选 $e=5$，然后由 $5d \equiv 1 \bmod 96$ 求 d（可以采用扩展欧几里得算法求出），解出 $d=77$。不难验证，$ed=5\times77=385=4\times96+1\equiv1 \bmod 96$。于是，公钥 $(e,n)=(5,119)$，而密钥 $d=77$。

现在对明文进行加密。设明文是 $x=19$。加密计算 $x^e=19^5=2\ 476\ 099\equiv66 \bmod 119$。66 就是对应于明文 19 的密文 y 的值。

用密钥 $d=77$ 进行解密时，计算 $y^d=66^{77}\equiv 19 \bmod 119$，即解密后应得出对应的明文 x 为 19。

3. RSA 算法安全分析

RSA 的安全性是基于大整数因子分解问题的难解性。如果 RSA 的模数 n 被成功分解为 p 和 q，则可算出 $\varphi(n)=(p-1)(q-1)$，进而通过计算 $d \equiv e^{-1} \bmod \varphi(n)$ 来获得私钥 d。密码分析者对 RSA 密码体制的一个明显的攻击手段是分解 n。因此，如果 RSA 密码体制是安全的，那么必须 $n=pq$ 是足够大的，使得分解它是计算上不可行的。

随着计算机计算能力的不断提高和分解算法的进一步研究，用于大因数分解的软件和硬件不断改善，开销越来越小，速度越来越快，这给 RSA 算法的安全性造成了很大威胁。512 位的 RSA 早已被证明是不安全的，而 1024 位 RSA 的安全性在几年之前也有人质疑。目前很多标准中都要求使用 2048 位的 RSA。

此外，关于 RSA 算法的很多种攻击并不是因为算法本身存在缺陷，而是由于参数选择不当造成的，为保证算法足够安全，参数须满足下面几个基本要求：要选择足够大的素数 p、q，使得 $|p-q|$ 较大，且 $(p-1)$ 和 $(q-1)$ 没有小的素因子；为加密实现方便，通常选择小的加密指数 e 且与 $\varphi(n)$ 互素，此时解密指数会较大；使用时不同用户不共用模数，且系统不能随意对信息解密（签名）。

为了提高加密速度，通常取 e 为特定的小整数，如 ISO/IEC 9796 甚至允许取 $e=3$。这样导致加密速度一般比解密速度快 10 倍以上。尽管如此，与对称密码体制相比（如 DES），RSA 的加、解密速度还是太慢，所以它很少用于数据的加密，而一般用于数字签名、密钥管理和认证方面。

2.3.2.2 SM2 椭圆曲线公钥密码算法

椭圆曲线公钥密码所基于的曲线性质如下。

(1) 有限域上椭圆曲线在点加运算下构成有限交换群,且其阶与基域规模相近。

(2) 类似于有限域乘法群中的乘幂运算,椭圆曲线多倍点运算构成一个单向函数。

在多倍点运算中,已知多倍点与基点求解倍数的问题称为椭圆曲线离散对数问题。对于一般椭圆曲线的离散对数问题,目前只存在指数级计算复杂度的求解方法。与大数因子分解问题及有限域上离散对数问题相比,椭圆曲线离散对数问题的求解难度要大得多。因此,在相同安全强度要求下,椭圆曲线密码较其他公钥密码所需的密钥规模小得多。鉴于其数学计算相对较为复杂,在此不予以详细介绍。

早在20世纪80年代,我国密码学者已经开始了椭圆曲线公钥密码算法的研究。2007年,国家密码管理局组织密码学专家成立专门的研究小组,开始起草我国自己的椭圆曲线公钥密码算法(简称SM2)标准。历时三年,完成了SM2算法标准的制定,2010年12月首次公开发布,2012年成为国家商用密码标准(标准号为GM/T 0003—2012),2016年成为中国国家密码标准(标准号为GB/T 32918—2016)。SM2为基于椭圆曲线密码的公钥密码算法标准,包含数字签名、密钥交换和公钥加密,其算法相关描述可登录http://gmssl.org/docs/sm2.html查看。

2.4 Hash 函数

Hash函数(哈希函数)也被称为杂凑函数、散列函数,是密码学的一个重要分支。Hash函数可以看作一种单向密码体制,即它是一个从明文到密文的不可逆映射,即只有"加密"过程,不能"解密"。Hash函数具有单向性和输出数据长度固定的特征,可以生成消息的"数字指纹"(也被称为消息摘要、杂凑值、Hash值或散列值),是实现数据完整性和身份认证的重要技术。

2.4.1 Hash 函数概述

Hash函数可以将"任意长度"的输入经过变换以后得到固定长度的输出。一般地,Hash值的生成过程可以表示为$h=\mathrm{H}(M)$,其中M是"任意"长度的消息,$\mathrm{H}(\cdot)$是Hash函数,h是固定长度的Hash值。

1. Hash 函数的性质

Hash函数应用于消息认证时,生成的Hash值作为消息的"指纹",要求其可以代表消息原文,因此必须具有以下性质。

(1) $\mathrm{H}(\cdot)$可以用于"任意"长度的消息。"任意"是指实际存在的。

(2) $\mathrm{H}(\cdot)$产生的Hash值是固定长度的。这是Hash函数的基本性质。

(3) 对于任意给定的消息x,容易计算$\mathrm{H}(x)$值。这是要求Hash函数的可用性。

(4) 单向性(抗原像性):对于给定的Hash值h,要找到M使得$\mathrm{H}(M)=h$在计算上是不可行的。

(5) 抗弱碰撞性(抗第二原像性):对于给定的消息M_1,要发现另一个消息M_2,满足$\mathrm{H}(M_1)=\mathrm{H}(M_2)$在计算上是不可行的。

(6) 抗强碰撞性:找任意一对不同的消息M_1、M_2,使$\mathrm{H}(M_1)=\mathrm{H}(M_2)$在计算上是不可行的。

(7) 消息对应Hash值的每个位应与消息的每个位有关联。当消息原文发生改变时,求

得的消息摘要必须相应变化。

Hash 函数的单向性、压缩性、抗碰撞性等特点使其能够应对一些认证问题，因此在数字签名、消息认证、口令的安全传输和存储等方面有非常广泛的应用。

2. Hash 函数的一般结构

到目前为止，Hash 函数的设计主要分为两类：一类是基于加密体制实现的，如使用对称分组密码算法的 CBC 模式来产生 Hash 值；另一类是直接构造复杂的非线性关系实现单向性。后者是目前使用较多的设计方法。

Hash 函数的一般结构如图 2.20 所示，也被称为迭代 Hash 函数结构。图中 ***IV*** 表示初始值，L 为输入分组数，CV_i 为链接变量，n 为 Hash 值的长度，M_i 为第 i 个输入分组，b 是输入分组的长度，f 是压缩函数。

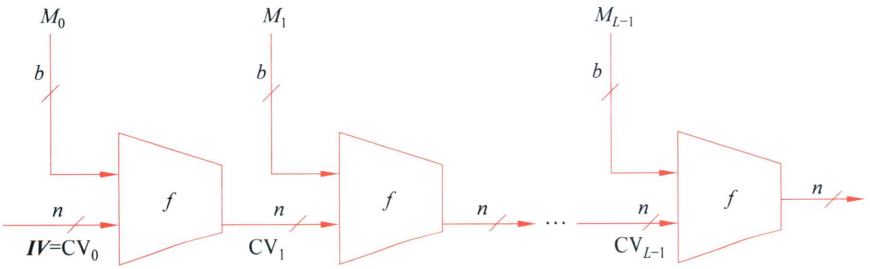

图 2.20 Hash 函数的一般结构

它由 Merkle 和 Damgård 分别独立提出，包括 MD5、SHA1 等目前所广泛使用的大多数 Hash 函数都采用这种结构。Hash 函数将输入消息分为 L 个固定长度的分组，每一分组长为 b 位，最后一个分组包含输入消息的总长度，若最后一个分组不足 b 位时，需要进行填充。由于输入包含消息的长度，所以攻击者必须找出具有相同散列值且长度相等的两条消息，或者找出两条长度不等但加入消息长度后散列值相同的消息，从而增加了攻击的难度。

该结构迭代使用一个压缩函数 f，压缩函数 f 有两个输入：一个是前一次迭代的 n 位输出，称为链接变量；另一个来源于消息的 b 位分组，并产生一个 n 位的输出。因为一般来说消息长度 b 大于输出长度 n，因此也被称为压缩函数。第一次迭代输入的链接变量又称为初值变量，由算法在开始时指定，最后一次迭代的输出即为 Hash 值。

攻击者对算法的攻击重点是压缩函数 f 的内部结构，由于压缩函数 f 和分组密码一样是由若干轮处理过程组成，所以对压缩函数 f 的攻击需通过对各轮之间的位模式分析来进行，分析过程常常需要先找出压缩函数 f 的碰撞。由于是压缩函数，其碰撞是不可避免的。因此，在设计压缩函数 f 时就应保证找出其碰撞在计算上是不可行的。

2.4.2 典型 Hash 函数算法

Hash 算法中比较著名的是 MD 系列和 SHA 系列。MD 系列是在 20 世纪 90 年代初由 MIT Laboratory for Computer Science 和 RSA Data Security Inc 的 Rivest 设计的，MD 代表消息摘要（Message Digest），MD2（1989 年）、MD4（1990 年）和 MD5（1991 年）都会产生一个 128 位的信息摘要。安全散列（Secure Hash Algorithm，SHA）系列算法是 NIST 根据 Rivest 设计的 MD4 和 MD5 开发的算法，美国国家安全局发布 SHA 作为美国政府标准。

1. MD 系列介绍

原始的 MD 算法从未公开发表过，第一个公开发表的是 MD2，接下来是 MD4 和 MD5。

Rivest 在 1989 年开发出 MD2 算法。在这个算法中，首先对信息进行数据补位，使信息的字节长度是 16 的倍数。然后，以一个 16 位的检验和追加到信息末尾，并且根据这个新产生的信息计算出散列值。

为了加强算法的安全性，Rivest 在 1990 年又开发出 MD4 算法。MD4 算法同样需要填补信息以确保信息的位长度减去 448 后能被 512 整除(信息位长度 mod 512=448)。然后，一个以 64 位二进制表示的信息的最初长度被添加进来。信息被处理成 512 位迭代结构的区块，而且每个区块要通过三个不同步骤的处理。研究人员很快发现了 MD4 版本中第一步和第三步的漏洞，利用一部普通的个人计算机在几分钟内可以找到 MD4 的碰撞。

于是，1991 年 Rivest 对 MD4 进行改进并设计了 MD5 算法，图 2.21 为 MD5 运算流程。MD5 算法比 MD4 算法复杂，并且速度较 MD4 快了近 30%，且在抗安全分析方面表现更好，因此在实际应用中受到欢迎。

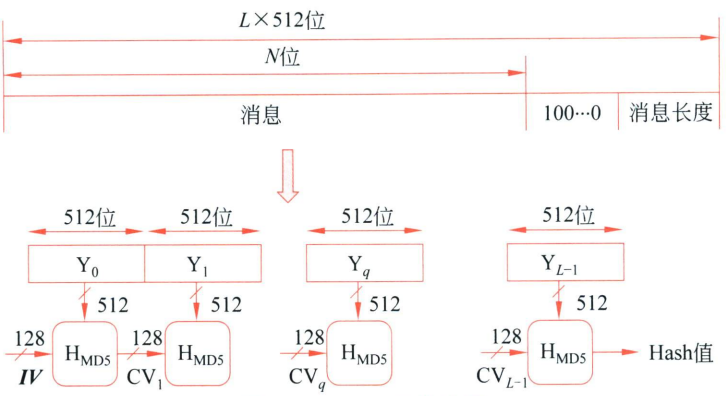

图 2.21 MD5 运算流程

2004 年 8 月 17 日，美国加州圣巴巴拉国际密码学会议上，山东大学的王小云教授做了破译 MD5、HAVAL-128、MD4 和 RIPEMD 算法的报告，公布了 MD 系列算法的破解结果，引发了密码学界的轰动。

2. SHA 算法介绍

NIST 于 1993 年开发的 Hash 算法为 SHA。两年之后，这个算法被修改为了今天广泛使用的形式。修改后的版本是 SHA-1，是数字签名标准中要求使用的算法。

SHA 接受任何有限长度的输入消息，并产生长度为 160 位的 Hash 值(MD5 仅仅生成 128 位的摘要)，因此抗穷举性更好。SHA-1 设计时基于和 MD4 相同的原理，它有 5 个参与运算的 32 位寄存器，消息分组和填充方式与 MD5 相同，主循环也同样是 4 轮，但每轮进行 20 次操作，非线性运算、移位和加法运算也与 MD5 类似，但非线性函数、加法常数和循环左移操作的设计有一些区别。

对于 Hash 函数，攻击者的主要目标不是恢复原始的明文，而是用非法消息替代合法消息进行伪造和欺骗，对 Hash 函数的攻击也是寻找碰撞的过程。在 MD5 被王小云院士为代表的我国专家破译之后，世界密码学界仍然认为 SHA-1 是安全的。2005 年 2 月 7 日，NIST 发表声明，称 SHA-1 没有被攻破，并且没有足够的理由怀疑它会很快被攻破，开发人员在 2010 年前应该转向更为安全的 SHA-256 和 SHA-512 算法。而仅仅在一周之后，王小云就宣布了破译 SHA-1 的消息。

因为 SHA-1 在美国等国家有更加广泛的应用，密码被破译的消息一出，在国际社会的反

响可谓石破天惊。换句话说，王小云院士团队的研究成果表明了从理论上讲电子签名可以伪造，必须及时添加限制条件，或者重新选用更为安全的密码标准，以保证电子商务的安全。

2001年，NIST公布SHA-2作为联邦信息处理标准，它包含6个算法：SHA-224、SHA-256、SHA-384、SHA-512、SHA-512/224、SHA-512/256。美国国家标准技术研究所2008年对国家标准进行更新，其中规定了SHA-1、SHA-224、SHA-256、SHA-384和SHA-512这几种Hash算法。SHA-1、SHA-224和SHA-256适用于长度不超过2^{64}二进制位的消息。SHA-384和SHA-512适用于长度不超过2^{128}二进制位的消息。目前已有多个网站提供了SHA系列各种Hash算法的在线使用功能。

NIST在2008年启动了SHA-3评选活动，2012年10月，Keccak函数族脱颖而出，被公布为新的Hash函数标准。SHA-3算法采用的是一种密封海绵结构，与多数Hash函数所采用的MD结构不同。它由特定的填充函数和置换函数Keccak-f组成，海绵结构如图2.22所示。P为输入，Z为输出，f为置换函数，r为分组长度，c为容量。海绵结构是针对一个固定置换f的迭代过程。

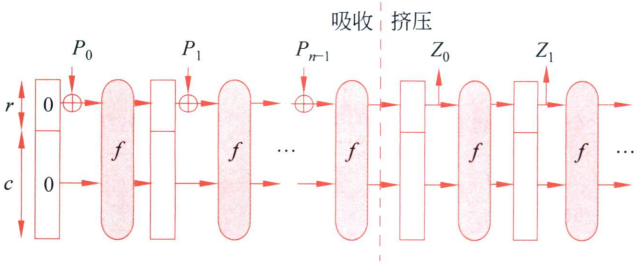

图2.22 海绵结构

海绵结构分为吸收和挤压两个阶段。第一阶段为吸收阶段，各消息块依次进入轮函数，此阶段不产生任何输出。当所有消息块吸收完毕后进入第二阶段，即挤压阶段，在此阶段根据需求可输出任意长度的摘要。

(1) 吸收阶段。输入消息经过填充，分为长度为r的各块($P_0, P_1, \cdots, P_{n-1}$)，每块分别与各次置换的输入状态的$r$长外部状态异或，而$c$长内部状态保持不变，形成作为本次置换$f$的输入。Keccak的填充方式为：首先添加一个1，之后添加数个0，最后添加一个1，即填充方式为10…01，其中0的个数应使填充后的长度是消息分组长度的最小整数倍。

(2) 挤压阶段。根据需要的输出长度，从各次置换的输出中分别提取$Z_0, Z_1, \cdots, Z_n$各子串，连接后形成算法输出。实际上海绵结构可以产生任意长度的输出。

按照NIST要求，算法输出长度分为224位、256位、384位和512位，Keccak算法的消息块长度r是根据输出长度变化的，即当输出长度为512位时，r为576位；当输出长度为384位时，r为832位；当输出长度为256位时，r为1088位；当输出长度为224位时，r为1152位。

3. SM3算法

2010年12月，国家密码管理局发布了SM3密码杂凑算法。《SM3密码杂凑算法》(GM/T 0004—2012)于2012年发布为密码行业标准，2016年发布为国家标准《信息安全技术 SM3密码杂凑算法》(GB/T 32905—2016)。2014年，我国提出将SM3密码杂凑算法纳入ISO/IEC标准的意见。2017年4月，SM3密码杂凑算法进入最终国际标准草案阶段，SC27工作组投票通过后将其正式列为ISO/IEC国际标准。2018年10月，含有我国《SM3密码杂凑算法》

的 ISO/IEC 10118-3:2018《信息安全技术杂凑函数第 3 部分：专用杂凑函数》第 4 版由 ISO 发布，《SM3 密码杂凑算法》正式成为国际标准。

SM3 密码杂凑算法基于 MD 结构，输出长度为 256 位，可用于数字签名、完整性保护、安全认证、口令保护及随机数生成等。在实现上，SM3 密码杂凑算法运算速率高、运用灵活、支持跨平台的高效实现，具有较好的实现效能。

2.4.3 Hash 函数应用

Hash 函数在数据完整性认证、数字签名、区块链等领域有广泛的应用，本节主要介绍 Hash 函数在消息认证码及区块链中的应用，数字签名相关内容将在 2.5 节进行描述。

1. 消息认证码

消息认证的目的主要包括：验证信息来源的真实性和验证消息的完整性。消息认证码(Messages Authentication Codes，MAC)是一种重要的消息认证技术，它利用消息和双方共享的密钥通过认证函数来生成一个固定长度的短数据块，并将该数据块附在消息后(图 2.23)。消息认证码是与密钥相关的 Hash 函数，也称消息鉴别码。消息认证码与 Hash 函数类似，都具有单向性，此外消息认证码还包括一个密钥。不同的密钥会产生不同的 Hash 函数，这样就能在验证发送者的消息有没有经过篡改的同时，验证是由哪个发送者发送的。

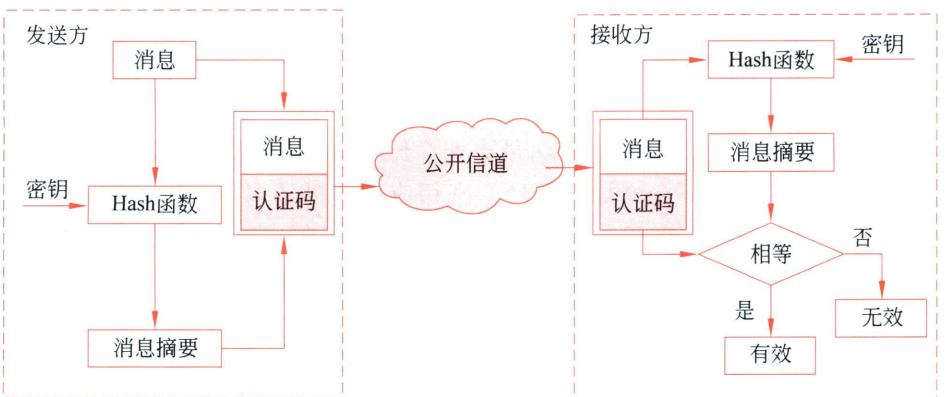

图 2.23 消息认证码的实现过程

MAC 算法与加密算法的不同之处为，MAC 不必是可逆的(一般为多到一的映射)，因此与加密算法相比更不易被攻破。上述过程中，由于消息本身在发送过程中是明文形式，所以这一过程只提供认证性而未提供保密性。为提供保密性可在生成 MAC 之后或之前进行一次加密，而且加密密钥也需被收发双方共享。人们通常希望直接对明文进行认证，因此先计算 MAC 再加密的使用方式更为常用。

近年来，人们越来越感兴趣于利用 Hash 函数来设计 MAC。然而，如 SHA-1 这样的 Hash 函数并不是专门为 MAC 设计的，由于 Hash 函数不依赖于密钥，所以它不能直接用于计算 MAC。目前，已经提出了许多方案将密钥加到现有的 Hash 函数中，其中 HMAC 是最受支持的方案，并且在 Internet 协议(如 SSL)中有应用。

HMAC 的实现过程如图 2.24 所示。其中，H 是一个嵌入的 Hash 函数；n 表示 Hash 值的长度；K 表示密钥，一般 K 的长度不小于 n，当使用长度大于 b 的密钥时，先用 H 对密钥进行计算，计算结果作为 HMAC 的真正密钥；K^+ 表示左边经填充 0 后的 K，K^+ 的长度为 b 位，L 表示 M 中的分组数；b 表示每个分组包含的位数；***IV*** 表示初始链接变量；ipad 表示 0x36 重

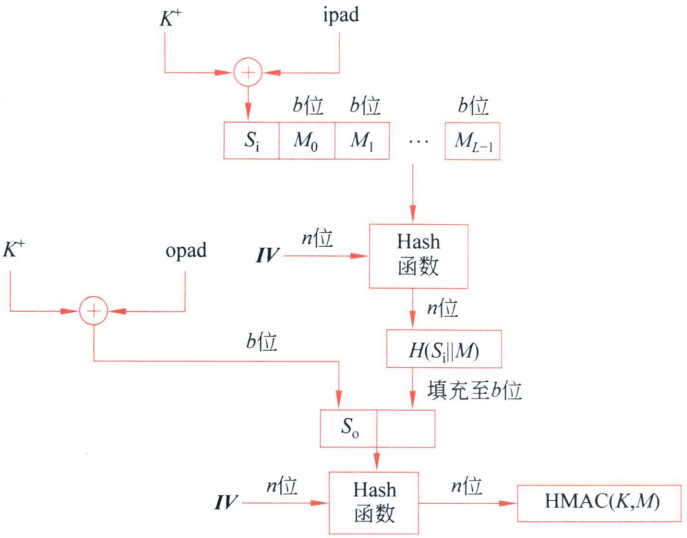

图 2.24 HMAC 算法实现过程

复 $b/8$ 次，opad 表示 0x5c 重复 $b/8$ 次。由此可知，HMAC 可表述为

$$\mathrm{HMAC}(K,M)=H((K^{+}\oplus\mathrm{opad})\parallel H((K^{+}\oplus\mathrm{ipad})\parallel M)))$$

需要强调的是 K^{+} 与 ipad 异或后，其信息位有一半发生变化；同样，K^{+} 与 opad 异或后，其信息位一半发生了变化。这两部分首先参与 Hash 运算，因此可以对其进行预计算，从而提高执行效率。

HMAC 的密钥长度可以是任意长度，最小推荐长度为 n 位，因为小于 n 位时会显著降低函数的安全性，大于 n 位也不会增加安全性。密钥应该随机选取，或者由密码性能良好的伪随机数产生器生成，且需定期更新。但如果密钥的随机性不好，则应当使用较长的密钥。

2. Merkle 树

Merkle 树（也称哈希树）是一种二叉树，由一个根节点、一组中间节点和一组叶节点组成。最下面的叶节点包含存储数据或其 Hash 值，每个中间节点是它的两个子节点内容的 Hash 值，根节点也是由它的两个子节点内容的 Hash 值组成，如图 2.25 所示。Merkle 树也可以推广到多叉树的情形。

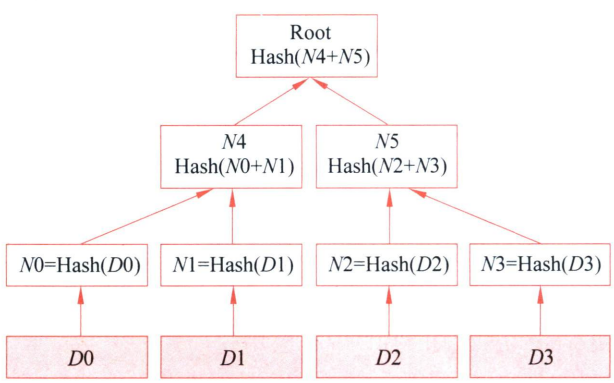

图 2.25 Merkle 树示例

Merkle 树的特点是，底层数据的任何变动，都会传递到其父节点，一直到树根。Merkle 树多数用来进行比对和验证处理，一般意义上来讲，它是 Hash 值大量聚集数据“块”的一种方

式，它依赖于将这些数据“块”分裂成较小单位的数据块。每个小单位数据块仅包含几个数据“块”，然后取每个小单位数据块再次进行 Hash 计算，重复同样的过程，直至剩余的 Hash 值总数仅变为 1，也就是根 Hash。

区块链系统中采用了 Merkle 二叉树，它的作用主要是快速归纳和校验区块数据的完整性，它会将区块链中的数据分组进行 Hash 运算，向上不断递归运算产生新的 Hash 节点，最终只剩下一个 Merkle 根存入区块头中，每个 Hash 节点总是包含两个相邻的数据块或其 Hash 值。使用 Merkle 树有诸多优点：首先是极大地提高了区块链的运行效率和可扩展性，使得区块链只需包含根 Hash 值而不必封装所有底层数据，这使得 Hash 运算可以高效地运行在智能手机甚至物联网设备上；其次是 Merkle 树可在不运行完整区块链网络节点的情况下对交易数据进行检验。所以，在区块链中使用 Merkle 树这种数据结构是非常有意义的。

2.5 数字签名

传统书信或文件是根据亲笔签名或印章来证明其真实性的，但在计算机网络中传送的报文又该如何盖章呢？这就是数字签名所要解决的问题。数字签名主要用于对数字消息进行签名，以防消息的冒名伪造或篡改，还可以用于通信双方的身份鉴别。

2.5.1 数字签名概述

随着计算机通信网络的迅速发展，特别是在大型网络安全通信中的密钥分配、认证及电子商务系统中，数字签名的使用越来越普遍，数字签名是防止信息欺诈行为的重要措施。

1. 数字签名的特点和功能

数字签名是电子信息技术发展的产物，是针对电子文档的一种签名确认方法，在数字系统中同样有签名应用的需求。例如，假定 A 发送一个认证的信息给 B，如果没有签名确认的措施，B 可能伪造一个不同的消息，但声称是从 A 处收到的；或者为了某种目的，A 也可能否认发送过该消息。就签名的本质而言，需要具有以下特点。

(1) 不可否认性，即必须可以通过签名来验证消息的发送者、签名日期和时间。

(2) 不可抵赖性，即必须可以通过签名对所签署消息的内容进行认证。

(3) 可仲裁性，即必须可以由第三方通过验证签名来解决争端。

在复杂而虚拟的网络环境中，数字签名与手写签名还存在不同之处，且很多方面是手写签名很难达到的。首先，签名的对象不同。手写签名的对象是纸质的文件，而数字签名的对象是传输在网络中的数字信息，是肉眼不可读的。其次，实现的方法不同。手写签名是将一串字符串附加在文件上，数字签名则是对整个消息进行某种运算，这一点在防篡改方面就凸显出数字签名的优势。数字签名与文件是一个整体，任何改动都会对整个签名结果产生影响，从而免去了手写签名需要对文件的每一页进行手签的烦琐劳动。因此数字签名技术可以更有效地防止文件的篡改。再次，验证的方式不同。手写签名的验证是通过和一个已有的签名进行对比，而模仿他人签名不是一件极其困难的事情，所以它的安全性得不到有效的保证。数字签名的验证则是通过一种公开的验证算法对签名进行计算，任何不一致都会被发现，因此具有很高的安全性。最后，在保证机密性方面，数字签名比手写签名更具有优势。因为数字签名可以实现对文件的加密，这样文件内容的机密性就得到了保证，而手写签名很难实现这一点。

数字签名是手写签名的数字模拟，但这种模拟不是简单的替代，尤其是当发送方和接收方

互相不完全信任的时候。数字签名在许多方面比手写签名更具有安全性，因此在电子政务、电子商务等重要场合中发挥着不可估量的作用。

综上所述，可以总结出数字签名具有以下功能。

（1）采用公钥的数字签名技术可以防范信息伪造。由于私钥由签名者秘密保管，所以由该私钥进行签名的文件可以表示该签名者的身份，任何其他人都不可能正确地伪造出该签名结果。

（2）在防范信息篡改方面，数字签名比手写签名更具有优势。假如有一份上百页的文件需要签署，为了保证文件不被篡改，需要在文件的每一页上进行签署，显然这样做很烦琐。数字签名技术可以使用户签名与文件成为一个整体，任何改动都会对签名结果产生影响。因此数字签名技术可以更有效地防止文件的篡改。

（3）在防范信息重放方面，数字签名具有很重要的作用。例如，在债务方面，数字签名可以防止债主重复利用一张收据对借款人进行勒索。因为数字签名可以对收据添加流水账号和时间戳等技术来有效防止重放攻击。

（4）数字签名可以有效防止签名者抵赖曾经签署过文件，从而防范抵赖。同时也要有相关措施防止接收者抵赖已经接收到了文件，可以要求接收者回送一个报文表明收到了文件，或者引入第三方仲裁机制。这样收发双方都无法抵赖曾经发送或接收过文件。

数字签名作为信息安全技术的基本工具，在网络安全，包括身份认证、数据完整性、不可否认性等方面有着重要应用。

2. 数字签名的原理

数字签名由公钥密码发展而来，与加密的不同之处在于：消息加密和解密可能是一次性的，它要求在解密之前是安全的；而一个签名的消息可能作为一个法律上的文件，如合同等，很可能在对消息签署多年之后才验证其签名，且可能需要多次验证此签名。

数字签名的目的是提供一种手段，使得一个实体把他的身份与某个信息捆绑在一起。一个消息的数字签名实际上是一个数，它依赖于签名者知道的某个秘密，也依赖于被签名信息本身。数字签名基于两条基本的假设：一是私钥是安全的，只有其拥有者才能获得；二是产生数字签名的唯一途径是使用私钥。

数字签名体制又被称为数字签名方案，一般由两部分组成，即签名算法和验证算法。签名算法或签名密钥是由签名者秘密保有的，而验证算法或验证密钥应当公开，以方便他人进行验证。一般来讲，数字签名方案包括三个过程：系统的初始化过程、签名生成过程和签名验证过程。

在系统的初始化过程中，需要产生数字签名所需要的基本参数，包括秘密的参数和公开的参数。这些基本参数为$(M,S,K,\mathrm{SIG},\mathrm{VER})$，其中，$M$ 代表明文空间，S 代表签名空间，K 代表密钥空间，SIG 为签名算法集合，VER 为验证算法集合。

在签名生成过程中，用户利用某种特定的算法对消息进行签名从而产生签名消息，这种签名方案可以是公开的也可以是私密的。该过程主要包含两个步骤：第一步，选取密钥；第二步，计算消息摘要，并对该摘要进行签名。

在签名验证过程中，验证者利用公开的验证方法对消息签名进行验证，从而判断签名的有效性。首先，验证者获得签名者的可信公钥；然后，根据消息产生摘要并对该摘要利用验证算法进行验证；最后，比较由验证算法计算出的消息与原始消息是否一致，若一致则该签名为有效，否则无效。

数字签名在具体实施过程中，发送方对信息进行数学变换，使所得信息与原始信息唯一对应；接收方进行逆变换，得到原始信息。只要数学变换优良，变换后的信息在传输过程中就具有很强的安全性，可以有效地防止干扰者的破译和篡改。该数学变换过程就是签名过程，通常对应某种加密措施；而在接收方的逆变换过程为验证过程，通常对应某种解密措施，如图 2.26 所示。

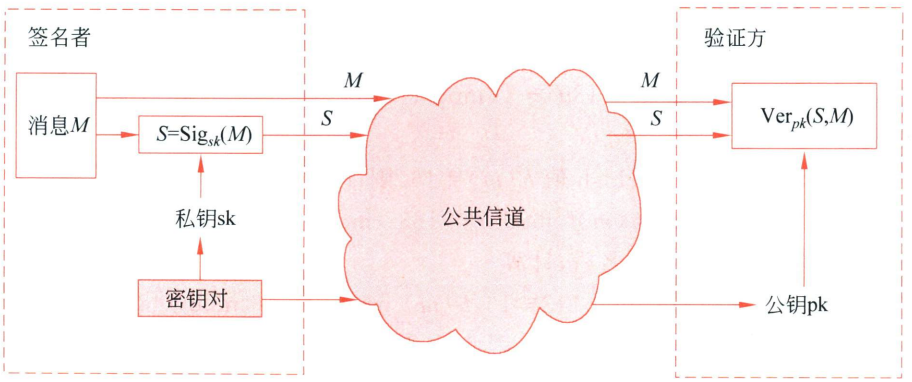

图 2.26 数字签名原理与过程

在传递签名时，通常要把签名附在原始消息之后一起传送给接收者。为了使签名方案在实际中便于使用，要求它的每一个签名算法 $\mathrm{Sig}_{sk} \in \mathrm{SIG}$ 和验证算法 $\mathrm{Ver}_{pk} \in \mathrm{VER}$ 都是多项式时间的算法。

对于数字签名技术在实现时还需要满足以下要求。

(1) 签名的产生必须使用签名者独有的一些信息以防伪造和否认，同时，要求保证独有信息的安全性。

(2) 签名的产生应较为容易。

(3) 签名的识别和验证应较为容易。

(4) 对已知的数字签名构造一新的消息或对已知的消息构造一假冒的数字签名在计算上都是不可行的。

2.5.2 典型数字签名算法

实现数字签名有很多种方法，可以基于对称密码体制，也可以依靠其共享密钥的保密性来实现数字签名，但其使用范围存在局限。目前，数字签名多数还是利用公钥密码体制来设计的。

1. 基于 RSA 的数字签名

RSA 签名方案是目前使用较多的一个签名方案，也是已经提出的数字签名方案中最容易理解和实现的签名方案，它的安全性是基于大整数因子分解的困难性。下面阐述 RSA 签名方案的实现过程。

1) 系统初始化过程

首先选取两个长度接近的大素数 p 和 q(推荐至少 1024 位)，计算 $n=pq$，其欧拉函数为 $\varphi(n)=(p-1)(q-1)$。然后随机选取整数 e $(1<e<\varphi(n))$，满足 $\gcd(e, \varphi(n))=1$。计算 d，满足 $de \equiv 1 (\mathrm{mod}\ \varphi(n))$。$n$ 公开，p 和 q 保密。e 为公钥，d 为私钥。

2) 签名生成过程

(1) 利用一个安全的 Hash 函数 h 来产生消息摘要 $h(m)$。

（2）用签名算法计算签名 $s=\mathrm{Sign}_{pk}(m)\equiv h(m)^d \bmod n$。

3）签名验证过程

（1）首先利用共享的 Hash 函数 h 计算消息摘要 $h(m)$。

（2）检验等式 $h(m) \bmod n \equiv s^e \bmod n$ 是否成立，若相等则签名有效，否则，签名无效。

RSA 数字签名算法实例。

系统初始化：假设发送者 A 选取 $p=13$，$q=11$，$e=13$，则有 $n=pq=143$，$\varphi(n)=(p-1)(q-1)=12\times10=120$。求解 $ed=13d\equiv 1(\bmod 120)$ 得 $d=37$。因此 A 的公钥为（$n=143$，$e=13$）；私钥为 $d=37$。

签名过程：假定消息 m 的 Hash 值 $h(m)=16$，则计算 m 签名

$$s=h(m)^d \bmod n\equiv 1637 \bmod 143\equiv 3$$

验证过程：接收者 B 收到签名后，计算

$$s^e \bmod n\equiv 313 \bmod 143\equiv 16,h(m) \bmod n\equiv 16 \bmod 143\equiv 16$$

等式 $h(m) \bmod n=s^e \bmod n$ 成立。因此，B 验证此签名有效。

RSA 签名方案中使用了 Hash 函数，使用这个函数比单纯对消息本身进行签名具有更好的抗攻击性。另外，对于大消息而言，对其 Hash 值的签名不仅不失数字签名特征，而且大幅提高了其签名和验证的效率。

2. DSA 数字签名

1994 年 12 月，美国国家标准和技术研究所正式颁布了数字签名标准 DSS（Digital Signature Standard）。DSS 最初建议使用 p 为 512 位的素数，q 为 160 位的素数，后来在众多的批评下，NIST 将 DSS 的密钥 p 从原来的 512 位增加到介于 512 位到 1024 位之间。当 p 选为 512 位的素数时，DSS 的签名长度为 320 位，这大大地减少了存储空间和传输带宽。

由于 DSS 具有较大的兼容性和适用性，因此 DSS 将得到广泛的应用。数字签名标准 DSS 中的算法常称为 DSA（Digital Signature Algorithm）。

1）系统初始化

选取一个素数 p，其中，$2^{511+64j}<p<2^{512+64j}$（$j\in\{0,1,\cdots,8\}$）；选取 $p-1$ 的一个 160 位的素数因子 q（$2^{150}<q<2^{160}$）；计算 $g=h^{(p-1)/q} \bmod p$，其中 $1<h<p-1$；生成一个随机数 x（$0<x<q$）；计算 $y=g^x \bmod p$。公钥为（p，q，g，y），私钥为 x。

2）签名生成

对明文 m 的签名算法如下。

（1）生成一个随机数 k（$0<k<q$）。

（2）计算 $r=(g^k \bmod p)\bmod q$。

（3）计算 $s=(k^{-1}(\mathrm{SHA}-1(m)+xr)) \bmod p$。式中：$\mathrm{SHA}-1(m)$ 是用 SHA－1 算法对明文 m 进行 Hash 运算；签名为（m，r，s）。

3）签名验证

对一个签名（m'，r'，s'）的验证过程如下。

（1）计算 $w=(s')^{-1}$。

（2）计算 $u_1=(\mathrm{SHA}-1(m')w) \bmod q$。

（3）计算 $u_2=(r'w)\bmod q$。

（4）计算 $v=((g^{u_1}y^{u_2})\bmod p) \bmod q$。

（5）检验 v 是否等于 r'。

只有当上述算法中 $v=r'$ 时，接收的签名才被验证正确。

DSA 算法是基于有限域上的离散对数问题设计的，DSA 算法不是标准的公钥密码，只能提供数字签名功能，但是由于具有良好的安全性和灵活性，被广泛应用于金融等领域。常见的数字签名算法还有 ElGamal、椭圆曲线数字签名算法等，另外还有一些特殊的数字签名算法，如盲签名、代理签名、群签名、门限签名等，它们与具体应用环境密切相关。

3. SM9 算法

SM9 标识密码算法由国家密码管理局 2016 年 3 月发布，为《SM9 标识密码算法》(GM/T 0044—2016)，共包含总则、数字签名算法、密钥交换协议、密钥封装机制和公钥加密算法、参数定义五部分。感兴趣的读者可查看 http://gmssl.org/docs/sm9-cn.html 了解算法具体内容。

这里简单介绍一下基于标识的密码算法。基于标识的密码算法是一门新兴的且正在不断快速发展中的公钥密码算法分支。这种密码算法的设计目标是让通信双方在不需要交换公私钥信息、不需要保存密钥的目录服务等基础设施、不需要使用第三方提供认证服务的情况下，保证信息交换的安全性并可以验证相互之间的签名。标识密码算法的设计思想是以实体的有效标识(如邮件地址、手机号码、QQ 号、身份证号码等)作为公钥，用户无须申请和交换证书，从而大大降低安全系统的复杂性。基于标识的密码体制假设存在一个可信的私钥生成器(Private Key Generator，PKG)作为系统的中心，当用户第一次加入系统，或达到系统要求的进行私钥更新的条件时，该中心会给用户生成一个私钥。该中心也被称为密钥生成中心，作用类似现实生活中的身份识别卡的发卡机构。用户自己选择标识信息，如用户姓名、用户的电子邮件、电话号码等信息或这些信息的组合作为其公钥使用，同时该用户也不能对代表自己的标识加以否认且该标识信息是可以公开发布的。而与用户选择的标识信息即公钥所对应的私钥则由密钥生成中心生成，像其他权威的身份识别卡的发卡机构一样，密钥生成中心须严格审查要申请私钥的用户身份，以避免非法用户盗用合法用户的标识/身份；同时密钥生成中心也应保护好计算用户私钥所使用的特权信息以防止用户私钥泄露。而对一般用户而言，则应防止自己的私钥泄露或其在使用时被非法复制。

与 1976 年 Diffie 和 Hellman 刚刚提出公钥密码学时的情况相似，虽然公钥密码学的设计思想被提出的同时也被人们看好，但是其具体的实现方案直到 1978 年才被研究出来。Shamir 提出基于标识的密码算法问题后，很多基于标识的加密方案被陆续提出，然而这些方案都不能完全令人满意。有的方案需要很强的硬件防篡改机制支持，有些计算时间过长没有实用价值，有些不能抵抗某些常见的攻击如用户共谋等。直到 2001 年，第一个真正实用的基于标识加密(也称基于身份加密，Identity-based Encryption，IBE)的方案由美国密码学家 Boneh 和 Franklin 设计出来。该方案很好地实现了 Shamir 的思想，即公钥可以是任意的字符串，使将用户标识/身份直接用于密码通信过程中的密钥交换、数字签名验证成为可能。

我国政府也非常重视标识密码算法的发展和应用，组织了相关领域的专家开展中国标识密码算法标准规范的制定工作，商用密码 SM9 算法于 2016 年 3 月由国家密码管理局正式对外公布。2017 年 10 月 30 日至 11 月 3 日，第 55 次 ISO/IEC 信息安全分技术委员会(SC27)会议在德国柏林召开。我国 SM2 与 SM9 数字签名算法经专家评审一致通过，成为国际标准，正式进入标准发布阶段，这也是本次 SC27 会议上密码与安全机制工作组通过的唯一进入发布阶段的标准项目。这两个数字签名机制是 ISO/IEC 14888—3/AMD1 标准研制项目的主体部分，也是我国商用密码标准首次正式进入 ISO/IEC 标准，极大地提升了我国在网络空间安全

领域的国际标准化水平。ISO/IEC 14888—3/AMD1 进入发布阶段，标志着我国在向 ISO 和国际电工委员会（International Electrotechnical Commission，IEC）贡献中国密码标准方面取得了重要突破，成功推进了我国数字签名标准在国际标准中的转化应用，对于增强我国密码产业在国际上的核心竞争力具有重要的意义。

2.6 密钥管理技术

现代密码体制要求密码算法是可以公开评估的，整个密码系统的安全性并不取决对密码算法的保密或是对密码设备等的保护，决定整个密码体制安全性的因素是密钥的保密性。密钥管理是密码学许多技术（如机密性、数据源认证、数据完整性和数字签名等）的基础，在整个密码系统中是极其重要的，密钥的管理水平直接决定了密码的应用水平。

2.6.1 密钥管理概述

密钥管理是处理密钥自产生到最终销毁的整个过程中的所有问题，包括密钥的生成、存储、分配/协商、使用、备份/恢复、更新、撤销和销毁等。密钥管理不仅影响系统的安全性，而且涉及系统的可靠性、有效性和经济性。

每个密钥都有其生命周期，有其自身的产生、使用和消亡的过程。密钥的产生、分配、存储、销毁等都是密钥管理的范畴。再好的密码算法，如果密钥管理出现问题，也很难保证系统的安全性。当然，密钥管理也涉及物理因素、人为因素及策略制度等方面的问题，本节主要关注理论和技术层面的一些基本知识。

由于应用需求和功能上的差异，在密码系统中所使用的密钥种类还是比较多的。按照加密内容的不同，密钥可以分为用于一般数据加密的密钥和用于密钥加密的密钥；按照所完成功能的差异，密钥可以分为用于验证数据签名的密钥（公钥）和用于实现数据签名的密钥（私钥）。根据不同种类密钥所起的作用和重要性不同，现有的密码系统的设计大都采用了层次化的密钥结构，这种层次化结构与对系统的密钥控制关系是对应的，图 2.27 展示了一个常用的（三级）简化密钥管理的层次结构。

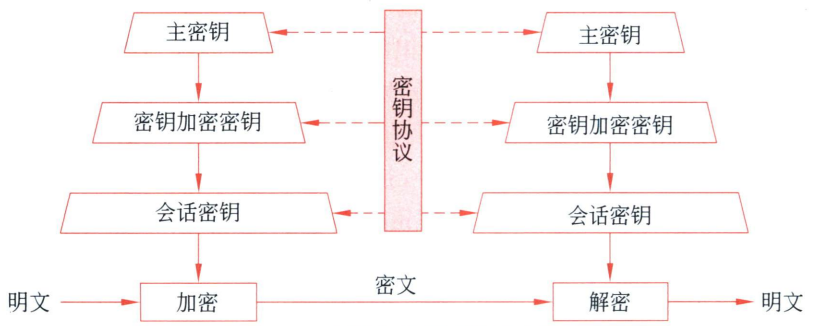

图 2.27 三级密钥管理的层次结构

一般情况下，按照密钥的生存周期、功能和保密级别可以将密钥分为 3 类：会话密钥、密钥加密密钥和主密钥。系统使用主密钥通过某种密码算法保护密钥加密密钥，再使用密钥加密密钥通过密码算法保护会话密钥，不过密钥加密密钥可能不止一个层次，最后会话密钥基于某种加解密算法来保护明文数据。在整个密钥层次体系中，各层密钥的使用由相应层次的密钥协议控制。

(1) 会话密钥：在一次通信或数据交换中用户之间所使用的密钥，是由通信用户之间进行协商得到的。

(2) 密钥加密密钥：用来对传输的会话密钥进行加密时采用的密钥，保护的对象是实际用来保护通信或文件数据的会话密钥。

(3) 主密钥：是由用户选定或由系统分配给用户的，可在较长时间内由用户所专有的秘密密钥，还可起到标识用户的作用。

密钥的分级系统大大提高了密钥的安全性。一般来说，越低级的密钥更换速度越快，最底层的密钥可以做到一次一换。在分级结构中，低级密钥具有相对独立性。一方面，它们被破译不会影响到上级密钥的安全；另一方面，它们的生成方式、结构、内容可以根据某种协议不断变换。

对于攻击者，密钥的分级系统意味着他所攻击的是一个动态系统。对于静态密钥系统，一份报文的破译就可能导致使用该密钥的所有报文的泄露。而对于动态密钥系统，由于低级密钥是在不断变化的，因而一份报文的破译造成的影响有限，且直接对主密钥发起攻击也是很困难的。因为，一方面，对主密钥保护是相当严格的，采取了各种物理手段；另一方面，主密钥的使用次数很少。

密钥的分级系统更大的优点还在于，它使得密钥管理自动化成为可能。对于一个大型密码系统而言，其需要的密钥数量是庞大的，都采用人工交换的方式来获得密钥已经不可能。在分级系统中，只有主密钥需要人工装入，其他各级密钥均可以由密钥管理系统按照某些协议来进行自动地分配、更换、撤销等。这既提高了工作效率，也提高了安全性。管理人员掌握着核心密钥，他们不直接接触普通用户使用的密钥与明文数据，普通用户也无法接触到核心密钥，这使得核心密钥的扩散面减到最小。

2.6.2 密钥建立技术

密钥的建立是信息安全通信中的关键问题，对安全通信的实现有着重要的影响。下面着重介绍会话密钥的建立方法。

2.6.2.1 对称密码体制的密钥建立

在对称密码体制下，必须通过安全可靠的途径将密钥送至接收端，系统的保密性取决于密钥的安全性。因此，对称密码体制中密钥的产生和密钥的管理是一个重要的研究课题，即如何产生满足保密要求的密钥，以及将密钥安全可靠地分配给通信对方。

按照是否需要第三方可信机构，其可分为无中心的密钥建立和有中心的密钥建立方式两类。无中心的密钥建立指用户直接将密钥传送给对方，此时参与者通常需要事先掌握一些资源。

1. 无中心的密钥建立

这里先介绍一个 Shamir 设计的无第三方参与的密钥建立协议。在协议过程中，用户 A 和 B 无须事先交换任何密钥，通过三次交互即可完成密钥传递，从而能够进行保密通信。该协议实现的前提是存在一种可交换的对称密码算法，即 $E_A(E_B(m))=E_B(E_A(m))$。协议过程描述如下。

(1) A 用自己的密钥加密 k 得到密文 $c_1=E_A(k)$，将密文 c_1 传送给 B。

(2) B 用自己的密钥加密 c_1 得到密文 $c_2=E_B(E_A(k))$，将密文 c_2 传送给 A。

（3）A用自己的密钥解密 c_2 得到 $c_3=D_A(E_B(E_A(k)))=D_A(E_A(E_B(k)))=E_B(k)$，将 c_3 传给B。

（4）B用自己的密钥解密 c_3 得到 k。

虽然这个协议可以保证密钥的正确性，但是由于没有提供身份认证，很容易在执行过程中发生冒充行为。因此，在使用此协议时，需要有其他配套协议提供身份认证。

2. 基于可信第三方的密钥建立

虽然已有协议可以在用户直接进行密钥建立，但是也存在一些问题。以点对点密钥建立为例，随着用户的增多，用户需要事先掌握的密钥加密密钥数量也大大增加，密钥的预分配问题很难解决。如果用户能和可信第三方（如密钥分配中心）之间建立共享密钥，那么可以借助可信第三方的帮助，在任何两个互不认识的用户之间建立一个共享密钥，这样无论系统有多少用户，预分配的密钥数量都是1。

假设可信第三方TTP提供密钥的产生、密钥的鉴别、密钥的分发等服务。发送者A和接收者B分别与可信第三方TTP共享一个密钥，A与TTP的共享密钥为 k_{AT}，B与TTP的共享密钥为 k_{BT}，A和B可以有两种途径建立密钥。

（1）用户选择共享密钥：A产生与B共享的密钥 k_{AB}，将密钥 k_{AB} 用A与TTP的共享密钥 k_{AT} 加密，然后把加密的结果 $E_{k_{AT}}(k_{AB})$ 传送给TTP。TTP接收到A发送的加密消息后，用与A共享的密钥 k_{AT} 解密后得到 k_{AB}，再用与B共享的密钥 k_{BT} 加密 k_{AB}，然后把加密的结果 $E_{k_{BT}}(k_{AB})$ 传送给B，或者把加密的结果传送给A再由A传给B。B使用与TTP共享的密钥 k_{BT} 解密后得到 k_{AB}，过程如图2.28所示。

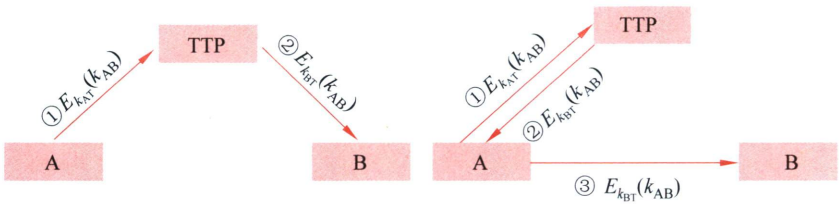

图2.28 用户选择共享密钥的密钥建立过程

（2）TTP选择共享密钥：A要求TTP产生密钥 k_{AB}，TTP产生密钥 k_{AB} 后分别使用与A共享的密钥 k_{AT} 和与B共享的密钥 k_{BT} 加密 k_{AB}，然后把加密的结果 $E_{k_{AT}}(k_{AB})$ 和 $E_{k_{BT}}(k_{AB})$ 分别传送给A和B，或者TTP把加密的结果都传送给A再由A传送给B。A和B分别使用与TTP共享的密钥 k_{AT} 和 k_{BT} 解密后得到 k_{AB}，如图2.29所示。

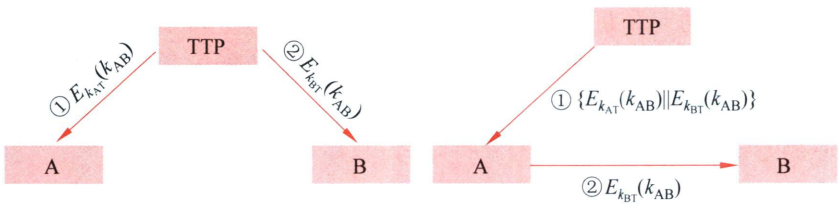

图2.29 TTP选择共享密钥的密钥建立过程

此处未介绍身份认证及防止重放、篡改等方面的技术，仅对基本思想、交互方式进行介绍。这些思想对设计会话密钥建立协议有指导意义，派生出很多重要的协议，如Kerberos密钥分发协议等。

2.6.2.2 公钥密码体制的密钥建立

公钥密码系统中，公钥是公开的。公钥的这种公开性为信息安全通信带来了深远的影响，同时也为攻击者提供了可乘之机。例如，攻击者可以用一个假公钥替换用户的真实公钥。因此，发展安全公钥密码系统的关键问题是如何确保公钥的真实性。本节将从密钥协商和公钥证书两方面来讨论针对公钥密码系统的密钥管理方法和技术。

公钥密码系统的一个重要应用是分配会话密钥，使两个互不认识的用户可以建立一个共享密钥，然后双方就可以利用该共享密钥保障通信的安全。例如：A 和 B 相互发送消息，A 首先建立一个共享密钥 key，并用 B 的公钥 k_e 加密 key 得到密文 $c=E(k_e,\text{key})$，然后把密文 c 传送给 B；接收方 B 用自己的私钥 k_d 解密密文 c 得到共享密钥 $\text{key}=D(k_d,c)$。最终，A 和 B 可以利用共享密钥 key 来保障双方会话的安全。在这种密钥建立的过程中，只有 A 对密钥的建立有贡献，B 只是被动地接收 A 发送的密钥。为了增加密钥的随机性，有时需要通信双方都对密钥的建立做出贡献。密钥协商就是这样的一种密钥建立方法。

Diffie-Hellman 密钥协商提供了对密钥分发的第一个实用的解决办法，使互不认识的双方通过公共信道交换信息建立一个共享的密钥。Diffie-Hellman 密钥协商是一种指数密钥交换，其安全性基于离散对数难解问题。

A 和 B 可以通过执行下面的协议建立一个共享密钥。假设 p 是一个足够大的素数，g 是模 p 的满足一定条件的元素（g 是 Z_p^* 中的本原根），p 和 g 是公开的。Diffie-Hellman 密钥协商协议过程如下。

(1) A 随机选择 a，满足 $1\leqslant a\leqslant p-2$，计算 $c=g^a \bmod p$ 并把 c 传送给 B。

(2) B 随机选择 b，满足 $1\leqslant b\leqslant p-2$，计算 $d=g^b \bmod p$ 并把 d 传送给 A。

(3) A 计算共享密钥 $k=d^a=g^{ab} \bmod p$。

(4) B 计算共享密钥 $k=c^b=g^{ab} \bmod p$。

此协议可以很容易扩展到多人的密钥协商，但是由于协议不包括通信方之间的身份认证过程，所以容易受到中间人攻击。为了抵抗这种攻击，在协议运行过程中需要结合认证技术。

公钥密码技术与对称密钥技术的最大区别就是：用公钥技术加密消息，通信双方不需要事先通过共享的安全信道协商密钥。加密方只要得到接收方的公开密钥就可以加密消息，并将加密后的消息发送给接收方。由于公钥是公开的，因此需要一种机制来保证用户得到的公钥是正确的，即需要保证一个用户的公钥在发布的时候是真实的，在发布以后不会被恶意篡改。公钥管理技术为公钥的分发提供了可信的保证。

2.6.3 公钥基础设施

公钥基础设施（Public Key Infrastructure，PKI）是网络安全的基础。其原理是利用非对称密码算法原理和技术所构建的，是用来解决网络安全问题的一种普遍适用的基础设施。有的学者把提供全面安全服务的基础设施，包括软件、硬件、人员和策略的集合称为 PKI。PKI 在网络信息空间的地位相当于电力基础设施在工业中的地位。可以说 PKI 是目前电子商务和电子政务必不可少的安全基础。

大多数公钥密码系统都容易受到中间人攻击。例如：考虑 A 和 B 进行通信的情况，假设 C 能够拦截公钥的交换，C 可以向 A 发送他自己的公钥，但故意将其表示成 B 的公钥；然后，他还可以向 B 发送自己的公钥，但故意表示成 A 的公钥。那么现在 C 便可以拦截 A 和 B 之间的所有通信了。如果 A 向 B 发送了一条加密的消息，由于实际上使用的是 C 的公钥，C 获

得消息后解密并进行存储，这样就可以稍后读取这条消息了。这之后，使用B的公钥加密其篡改后的消息，继续将其发送给B。B获得消息后能够为其译码，但不知道它实际上是来自C而非A。

上面的问题的实质是A没办法确定其得到的密钥是否真的属于B。公钥是公开的，因此无须确保秘密性。然而，却必须确保公钥的真实性和完整性，绝对不允许攻击者替换或篡改用户的公钥。如果公钥的真实性和完整性受到危害，则基于公钥的各种应用的安全将受到危害。这实际上是一种信任问题。A和B可以相互信任，但他们如何能够知道与其通信的人到底是不是对方所声称的人呢？他们如何才能确保所收到的公钥真正属于他们要发消息的人呢？在一个小的团体内部，这个问题很容易解决，但在互联网这样的大环境中，显然需要建立一种信任机制。

这就是PKI要解决的核心问题。它是一种可信的第三方，保持着对每个用户密钥的跟踪，其两种基本操作是：证明（将公钥值与所有者绑定的过程）和验证（验证证书依然有效的过程）。当前，PKI已经成为一种用公钥概念和技术实施、提供安全服务的具有普适性的安全基础设施，以核心的密钥和证书管理服务为基础，PKI及其相关应用保证了网上数字信息传输的保密性、完整性、真实性和不可否认性。

在电子商务应用环境中，交易双方互不隶属，仅仅依靠交易双方无法实现信任凭证，必须要依靠一个交易双方都认可的可信第三方机构来提供信任证明，PKI便提供了第三方信任机制。但是，特别需要引起注意的是，社会活动中还有很多不需要引入第三方信任的场合，如在一个单位的内部。在这种情况下，虽然也需要使用证书与公钥技术，但不必引入PKI架构，否则有可能带来巨大的资源浪费。

2.6.3.1 PKI的组成和功能

从广义上讲，PKI体系是一个集网络建设、软硬件开发、网络安全技术、策略管理和相关法律政策于一体的大型的、复杂的、分布式的综合系统。在这里仅讨论狭义范围的PKI。一般而言，一个比较完整的PKI至少应包括以下7部分的内容。

1. 认证机构

PKI的核心是信任关系的建立和管理。假设甲国公民A和乙国公民B互相不认识，更不信任对方，如果存在公正的可信任的第三方C（如护照签发机关），使A和B都直接信任C，那么此时公民A和B就可以信任对方了，这就是所谓的第三方信任。由此可以看出，在建立第三方信任时，公正、可信任的第三方C对于信任关系的建立和巩固起到至关重要的作用。而认证机构就扮演着一个具有权威性的第三方角色，是PKI的主要组成部分之一，它的核心职责就是完成证书的管理（证书的概念请参考后面的介绍）。认证机构使用自己的私钥对证书注册机构提交的证书申请进行签名，来保证证书数据的完整性，任何对证书内容的非法修改，用户都会使用认证机构的公钥进行验证，这是证书合法性的基础。

广义的认证中心还应包括证书注册机构，它是数字证书申请注册、签发和管理的机构，是认证机构和最终用户之间的接口。这项功能通常由人工完成，也可以由机器自动完成。在实际应用中，有些PKI的证书注册机构功能并不独立存在，而是合并在认证机构之中。

对认证机构来说最重要的事情是对自己的一对密钥的管理，认证机构的私钥必须高度保密，以防止他人伪造证书。认证机构的公钥在网上公开，因此整个网络系统必须保证完整性。认证机构在为用户颁发数字证书时，用户的公钥有两种产生的方式：一是用户自己生成密钥对，然后将公钥以安全的方式传送给认证机构，该过程必须保证用户公钥的验证性和完整性；

二是认证机构替用户生成密钥对，然后将其以安全的方式传送给用户。该方式下由于用户的私钥为认证机构产生，所以对认证机构的可信性有更高的要求。

2. 证书和证书库

证书是数字证书或电子证书的简称，是构成 PKI 的基本元素。它是参与网上信息交流及商务交易活动的各个实体（如持卡人、企业、商家、银行等）的身份证明，证明该用户的真实身份和公钥的合法性，以及该用户与公钥的匹配关系。它相当于护照，而且是一种“电子护照”。

证书库是网上的一种公共信息存储仓库，用于存储、撤销认证机构已签发证书及公钥，可供公众进行开放式查询。一般而言，查询的目的有两个：一是获得商务活动时对方的公钥，以便加密数据并通过网络传送，完成商务活动；二是验证对方的证书是否已经作废，即该证书是否已经不再被使用。

3. 密钥备份及恢复系统

密钥备份及恢复是 PKI 密钥管理的重要内容之一。如果某用户由于某种原因不慎丢失解密密钥，意味着加密数据的完全丢失，那么就有可能造成合法的数据大量丢失，导致不可挽回的巨大经济损失。为了避免灾难的发生，PKI 提供了密钥备份及恢复系统。当用户证书生成的同时，解密密钥就被认证机构备份并存储起来，当需要恢复时，用户只需要向认证机构提出申请，认证机构就会为用户自动进行恢复。当然，签名私钥为确保其唯一性则不能够做备份和恢复。

4. 密钥和证书的更新系统

与日常生活中使用的各种各样的身份证件相似，证书也有自己的使用期限，而且由于某种原因，证书在有效期内也可能作废，如密钥丢失、用户的个人身份信息发生改变、认证机构对用户不再信任或用户对该认证机构不再信任等各种情况。为此，证书和密钥必须保持一定的更新频率。证书的更新一般可以有 3 种方式：更换一个或多个主题的证书；更换由某一对密钥签发的所有证书；更换某一个认证机构签发的所有证书。即使在用户正常使用证书的过程中，PKI 也会自动不定时到目录服务器中检查证书的有效期，当有效期将满时，认证机构会自动启动更新程序，将旧证书列入作废证书列表（俗称黑名单），同时生成一个新证书来代替原来的旧证书，并通知用户。

5. 证书历史档案

经过若干时间以后，每一个用户都会形成多个旧证书和一个当前证书。这些旧证书及相应的私钥就组成了用户密钥和证书的历史档案。记录整个密钥历史是非常重要的。例如，某用户几年前用自己的公钥加密的数据，或者其他人用自己的公钥加密的数据就无法用现在的私钥解密，那么该用户就必须提出申请，从他的密钥历史档案中，查找到当年使用的私钥来解密这些数据，保证数据使用的连贯性。

6. 应用接口系统

为方便用户操作，解决 PKI 的应用问题，一个完整的 PKI 还必须提供良好的应用接口系统，以实现数字签名、加密传输数据等安全服务，使各种应用能够安全、一致、可信地与 PKI 交互，确保安全网络环境的完整性、易用性和可信度。

7. 交叉认证

交叉认证是为了解决公共 PKI 体系中各个认证机构互相分割、互不关联的“信任孤岛”问题，实现多个 PKI 域之间互联互通，从而满足安全域可扩展性的要求。目前，比较典型的交叉认证的模型有：树状认证模型、网状认证模型、桥式模型、信任列表模型、相互承认模型等。

2.6.3.2 数字证书

在PKI系统中，数字证书简称为证书。它是一个数据结构，是一种由一个可信任的权威机构签署的信息集合。PKI系统中的公钥证书是一种包含持证主体标识、持证主体公钥等信息，并由可信任的认证机构签署的信息集合，主要用于确保公钥与用户的绑定关系，它的持证主体可以是人、设备、组织机构或其他主体。

任何一个用户只要知道认证机构的公钥，就能检查证书签名的合法性。如果检查正确，那么用户就可以相信那个证书所携带的公钥是真实的，而且这个公钥就是证书所标识的那个主体的合法的公钥。

由于PKI适用于异构环境中，所以证书的格式在所适用的范围内必须统一。目前应用最广泛的证书格式是国际电信联盟(ITU)提出的X.509版本3格式。X.509标准最早于1988年颁布，在此之后又于1993年和1995年进行过两次修改，此后，互联网工程任务组(The Internet Engineering Task Force，IETF)针对X.509在互联网环境的应用，颁布了一个作为X.509子集的RFC 2459，从而使X.509在互联网环境中得到广泛应用。

一个标准的X.509数字证书包含以下内容。

(1) 证书的版本信息。

(2) 证书的序列号，每个证书都有一个唯一的证书序列号。

(3) 证书所使用的签名算法。

(4) 证书的发行机构名称，命名规则一般采用X.500格式。

(5) 证书的有效期。

(6) 证书所有人的名称，命名规则一般采用X.500格式。

(7) 证书所有人的公开密钥。

(8) 证书发行者对证书的签名。

X.509证书的标准格式如图2.30所示。

我国于2006年8月发布了国家标准《信息安全技术公钥基础设施数字证书格式》，这一标准主要根据RFC 2459制定，并结合我国数字证书应用的特点进行了相应的扩充和调整。

2.6.3.3 PKI应用

PKI技术的广泛应用能满足人们对网络交易安全保障的需求。当然，作为一种基础设施，PKI的应用范围非常广泛，并且在不断发展之中，下面以安全电子邮件场景为例，介绍PKI的应用。

1. S/MIME

电子邮件方便快捷的特性已使其成为重要的沟通和交流工具，但目前的电子邮件系统却存在着较大的安全隐患：邮件内容以明文形式在网上传送，易遭到监听、截取和篡改；无法确定电子邮件的真正来源，也就是说，发信者的身份可能被人伪造。因此，安全电子邮件协议——安全/多用途互联网邮件扩展(Secure Multipurpose Internet Mail Extension，S/MIME)应运而生。S/MIME为电子邮件提供了数字签名和加密功能，其实现需要依赖于PKI技术。

目前，S/MIME已经被广泛应用于各种客户端和平台，大多数电子邮件客户端软件都支持S/MIME协议的最新版本，因此大多数不同的电子邮件客户程序彼此之间都可以收发安全电子邮件。

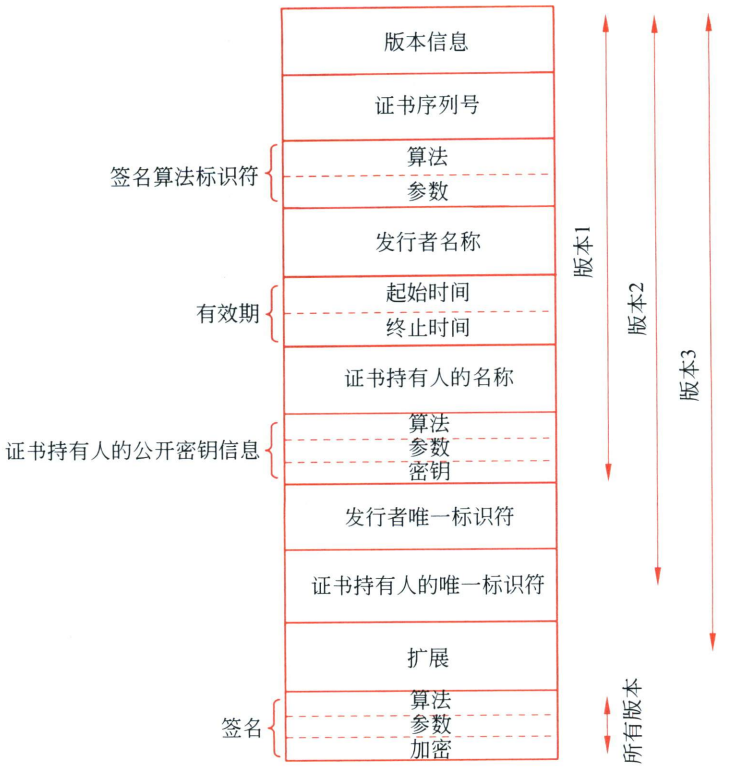

图 2.30 X.509 证书的标准格式

2. Outlook Express

Outlook Express 是户常用的客户端电子邮件收发软件，能够自动查找安装在计算机上的数字证书，将这些证书同邮件账号相绑定，并自动将别人发送的数字证书添加到通信簿中。

在使用 Outlook Express 安全电子邮件功能之前，用户需要有自己的数字标识。数字标识可以从认证机构那里获得，它包含一个私钥（该私钥一直保留在用户的计算机上），以及一个证书（含有公钥）。

为了发送签名邮件，用户必须有自己的数字证书；为了发送加密电子邮件，用户需要有收件人的数字证书。获得收件人数字证书的方法是让对方发送带有其数字签名的邮件，将该邮件打开后，在右边会看到对方的证书标志，单击该标识，找到“安全”项，单击“查看证书”按钮，可以查看“发件人证书”；单击“添加到通信簿”按钮后，在通信簿中保存发件人的加密首选项，这样对方数字证书就被添加到通信簿中了。有了对方的数字证书后，就可以向对方发送加密邮件。

3. PGP

Pretty Good Privacy（PGP）是 1990 年左右由菲利普·齐墨尔曼个人编写的密码软件，作为最广泛使用的保障电子邮件安全的技术之一，主要提供机密性、完整性和认证服务，用于保障电子邮件和文件存储的安全。PGP 的功能包括认证、机密性、压缩、电子邮件的兼容性、分段和组装及密钥管理。

PGP 被广泛用来加密重要文件和电子邮件，以保证它们在网络上的安全传输，或为文件做数字签名防止被篡改或伪造。PGP 可以在 DOS/Windows、UNIX、OS/2、Linux 等很多平台上运行。PGP 的使用说明可以参见 http://www.pgp.cn/Tutorial/。此外，还有一个由 GUN 遵照 OpenGPGP（RFC 4880）规范编写的称为 GnuPG 的软件。

PGP 建立在一些经过公开评议、被认为非常安全的算法的基础上。例如，PGP 中包括了 RSA、DSS、EIGamal、ECC 和 Diffie-Hellman 等公开密钥加密算法，CAST-128、IDEA、3DES、Blowfish、SAFER-SK128 和 AES 等常规加密算法，MD5、SHA-1、RIPE-MD/160、MD2、TIGER/192 等 Hash 函数算法，ZIP、ZLIP 等压缩算法。它的应用范围非常广。从公司到个人、从邮件到文件都可以使用。它也是一个非常国际化的软件，其程序和文档被译成了多种语言。PGP 的版本有两大类：PGP 免费版（仅供个人用于非商业目的）和 PGP 商业版（公司或企业用），使用起来非常简单方便。它不是任何政府或标准化组织开发的，因此不受它们的控制。到目前为止，PGP 已在全球赢得众多的支持者，且已经成为基于 Internet 的电子邮件加密工业标准。

2.7 本章小结

本章围绕一些主要的密码算法及其应用进行介绍，以密码技术概述为切入点，重点介绍了分组密码、序列密码、公钥密码、Hash 函数、数字签名、密钥管理中的基础知识和国内外典型算法，并以安全电子邮件为例介绍了密码技术的综合应用。

通过本章内容的介绍，可以了解到保证数字信息机密性的最有效方法是使用密码算法对其进行加密；保证信息完整性的有效方法是利用 Hash 函数计算信息“指纹”，实现完整性检验；保证信息认证性的有效方法是密钥和 Hash 函数结合来确定信息的来源；保证信息不可抵赖性的有效方法是对信息进行数字签名。此外，利用密码机制及密钥管理技术可以有效地控制信息，使信息系统只为合法授权用户所用。

密码学是保障信息安全的核心，信息安全是密码学研究与发展的目标。近年来，我国高度重视密码研究、应用及密码算法管理工作，国家密码管理局发布了多个商用密码算法，形成了完整、自主的国产商用密码算法体系，在保障我国信息安全中发挥了巨大作用。学习和了解密码学知识不仅有助于保护自身权益和隐私，还有助于对一些前沿科技的理解和应用。尽管密码学对于安全通信是基础必要的，对于保障信息安全的作用是十分重要的，但仅仅依靠密码技术还是远远不够的。

思考题

1. 简述密码体制的组成部分及其分类。
2. 分别说明对称密码体制和非对称密码体制的优点和不足。
3. 简述 Shannon 提出的设计密码体制的两种基本方法。
4. 简述两种常见的分组密码结构。
5. 简述分组密码的工作模式。
6. 简述序列密码与分组密码的不同之处。
7. 简述单向陷门函数的含义。
8. 简述 Hash 函数应具有的性质。
9. 简述数字签名与手写签名的不同之处。
10. 简述密钥管理的层次结构。
11. 简述 Diffie-Hellman 密钥协商协议过程。